

UNIVERSIDAD DE SANTIAGO DE
COMPOSTELA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA

Consulta eficiente de redes geoespaciales

Una comparativa entre BBDD de grafos y
objeto-relacionales

Autor:

Daniel Ramos Rodríguez

Tutor:

José Ramón Ríos Viqueira

Grado en Ingeniería Informática

Febrero 2026

Trabajo de Fin de Grado presentado en la Escuela Técnica Superior de
Ingeniería de la Universidad de Santiago de Compostela para la obtención del
Grado en Ingeniería Informática



D. José Ramón Ríos Viqueira, profesor do Departamento de Electrónica e Computación da Universidade de Santiago de Compostela

INFORMA:

Que a presente memoria, titulada *Consulta eficiente de redes geoespaciales: Una comparativa entre BBDD de grafos y objeto-relacionales*, presentada por **D. Daniel Ramos Rodríguez** para superar os créditos correspondentes ao Traballo de Fin de Grao da titulación de Grao en Enxeñaría Informática, realizouse baixo a súa titoría no Departamento de Electrónica e Computación da Universidade de Santiago de Compostela.

E para que así conste aos efectos oportunos, expide o presente informe en Santiago de Compostela, a 13 de xaneiro de 2026:

Titor,

Firmado digitalmente por
RIOS VIQUEIRA, JOSE
RAMON (FIRMA)
Nombre de reconocimiento
(DN): c=ES,
serialNumber=44807701K,
sn=RIOS, givenName=JOSE
RAMON, cn=RIOS VIQUEIRA,
JOSE RAMON (FIRMA)
Fecha: 2026.01.27 16:14:05
+01'00'

José Ramón Ríos Viqueira

Alumno,

Daniel Ramos Rodríguez

Resumen

La gestión eficiente de las redes de transporte público presenta desafíos progresivamente mayores debido a la necesidad de integrar grandes volúmenes de datos tanto espaciales como temporales. Ante este escenario, surge la necesidad de evaluar si las arquitecturas SQL tradicionales siguen siendo óptimas, o por el contrario, los paradigmas orientados a grafos ofrecen ventajas significativas.

En este trabajo se han modelado dichas redes geoespaciales en dos sistemas gestores de bases de datos diferentes, PostgreSQL y Neo4J, partiendo del estándar GTFS. Después, se ha desarrollado un conjunto representativo de consultas que responden a necesidades de información reales, tanto de usuarios de la red como de las empresas que ofrecen dichos servicios.

A continuación, se ha trabajado en la representación gráfica, sobre el globo terráqueo, de los resultados generados, utilizando QGIS y Folium, y, en último lugar, se ha llevado a cabo una comparativa entre las prestaciones de ambos sistemas, centrada no solo en la eficiencia computacional, sino también en la facilidad de modelado y expresión de consultas.

Los resultados obtenidos muestran que PostgreSQL ofrece un rendimiento superior en ingesta de datos y agregación masiva de estos, contando además con excelente soporte para información geoespacial. A su vez, Neo4J destaca en consultas que requieren navegación desde un número reducido de nodos iniciales, y ofrece una sintaxis más legible y fácil de mantener, aunque presenta limitaciones en el soporte de bibliotecas.

Índice general

1	Introducción	1
2	Especificación de requisitos	3
2.1	Descripción de dominio y datos	3
2.2	Alcance	4
2.3	Requisitos funcionales	5
2.4	Requisitos no funcionales	11
2.5	Requisitos de información	12
3	Diseño	13
3.1	Arquitectura del sistema	13
3.2	Infraestructura y despliegue	14
3.2.1	Aislamiento y reproducibilidad	14
3.2.2	Conexiones	15
3.2.3	Entorno de ejecución	16
3.3	Principios y patrones de diseño	17
3.3.1	Separación de responsabilidades	17
3.3.2	KISS (<i>Keep it simple, stupid</i>)	17
3.3.3	DRY (<i>Don't repeat yourself</i>)	17
3.3.4	Defensa en profundidad	17
3.3.5	Inyección de dependencias	18
3.3.6	Fachada	18
3.4	Datos	19
3.4.1	Ciclo de vida de la información	19
3.4.2	Modelo relacional (PostgreSQL)	19
3.4.3	Modelo orientado a grafos (Neo4J)	20
3.5	Algoritmos y consultas	22
3.5.1	Lógica declarativa	22
3.5.2	Enrutamiento híbrido	24
4	Pruebas	29
4.1	Estrategia de pruebas	29
4.1.1	Alcance	29

4.1.2	Herramientas e infraestructura	30
4.1.3	Conjuntos de datos	30
4.1.4	Metodología	31
4.2	Integridad de los datos	32
4.3	Algoritmos de consulta	32
4.3.1	Casos de prueba aleatorizados	34
4.3.2	Valores límite	34
4.3.3	Invariantes	35
5	Evaluación comparativa	37
5.1	Eficiencia	37
5.1.1	Tiempo de importación (input)	37
5.1.2	Tiempo de consulta (output)	38
5.2	Modelado y expresión	39
5.3	Soporte de bibliotecas	42
5.4	Resultados	42
6	Conclusiones y posibles ampliaciones	43
6.1	Hallazgos	43
6.2	Grado de cumplimiento de los objetivos	44
6.3	Posibles vías de mejora	45
6.3.1	Soporte para GTFS Realtime	45
6.3.2	Análisis de topología y resiliencia de la red	45
6.3.3	Enrutamiento con restricciones multicriterio	45
A	Manual técnico	47
A.1	Código fuente	47
A.2	Requisitos, dependencias e instalación	47
A.3	Estructura del proyecto	48
A.4	Modificaciones	49
A.4.1	Actualizaciones en el esquema	49
A.4.2	Adición de consultas	49
A.5	Ejecución de pruebas	50
A.6	Errores frecuentes	50
A.6.1	Las pruebas producen errores de conexión	50
A.6.2	Otros errores	50
B	Manual de usuario	51
B.1	Especificaciones	51
B.2	Dependencias	51
B.3	Instalación	52
B.4	Consultas	53
B.4.1	Queries predefinidas	53

B.4.2	PostgreSQL	53
B.4.3	Neo4J	55
B.5	Visualización sobre mapa	57
B.5.1	QGIS	57
B.5.2	Folium (Python)	58
B.6	Errores frecuentes	60
B.6.1	No es posible conectar con el daemon de Docker	60
B.6.2	No se puede establecer comunicación con un contenedor Docker	60
B.6.3	Las consultas en Neo4J no producen resultados	60
B.6.4	Los scripts en Python fallan al tratar de importar bibliotecas	61
C	Licencia	63
	Bibliografía	65

Índice de figuras

3.1	Diagrama de la arquitectura general del sistema.	14
3.2	Ciclo de vida de las conexiones del sistema.	15
3.3	<i>Pipeline</i> de defensa en profundidad.	18
3.4	Modelo entidad-relación (simplificado) seguido en PostgreSQL.	20
3.5	Meta-grafo (simplificado) seguido en Neo4J.	21
3.6	Ejemplo de funciones ventana en PostgreSQL.	23
3.7	Ejemplo de procesamiento de colecciones en Neo4J.	23
3.8	Código de invocación de una consulta predefinida en Neo4J.	24
3.9	Implementación simplificada del algoritmo CSA híbrido.	27
4.1	Muestra de código de las pruebas de importación (simplificada).	33
4.2	Ejecución de las pruebas de importación (<i>dataset</i> de Galicia).	33
4.3	Muestra de código de las pruebas de valores límite (simplificada).	35
4.4	Muestra de código de las pruebas de invariantes (simplificada).	36
4.5	Ejecución de las pruebas de consultas (<i>dataset</i> de Galicia).	36
5.1	Tiempos de importación en PostgreSQL y Neo4J.	38
5.2	Tiempos de consulta en PostgreSQL y Neo4J.	39
5.3	Comparativa de modelado entre PostgreSQL (izq.) y Neo4J (der.).	40
5.4	Código de <code>overlapping_segments</code> en PostgreSQL.	41
5.5	Código de <code>overlapping_segments</code> en Neo4J.	41

B.1	Ejemplo de consulta en PostgreSQL.	56
B.2	Interfaz web de Neo4J.	56
B.3	Visualización de múltiples resultados en QGIS.	59
B.4	Visualización de un análisis de alcanzabilidad en Folium.	59

Índice de cuadros

2.1	Principales entidades en el estándar GTFS.	4
4.1	Conjuntos de datos utilizados para las pruebas.	31
5.1	Resumen comparativo: PostgreSQL frente a Neo4J.	42
B.1	Lista de dependencias software.	52
B.2	Consultas disponibles en ambos sistemas.	54
B.3	Consultas exclusivas de PostgreSQL.	55

Índice de requisitos

RF-01	Importación de datos	5
RF-02	Validación de integridad y restricciones	5
RF-03	Servicios de transporte activos	5
RF-04	Estadísticas diarias	6
RF-05	Horas de salida de trayectos	6
RF-06	Tiempos de espera en la red	6
RF-07	Próximas salidas	7
RF-08	Segmentos con solapamiento	7
RF-09	Ranking de rutas por relevancia	7
RF-10	Ranking de rutas por velocidad	7
RF-11	Paradas cercanas	8
RF-12	Paradas más transitadas	8
RF-13	Histograma de comienzo de trayectos	8

RF-14	Mapa de calor de paradas	9
RF-15	Análisis de alcanzabilidad	9
RF-16	Camino más corto	9
RF-17	Visualización espacial de resultados	10
RF-18	Generación de gráficas	10
RF-19	Medición de rendimiento	10
RNF-01	Escalabilidad respecto al conjunto de datos	11
RNF-02	Extensibilidad del catálogo de consultas	11
RNF-03	Portabilidad de las visualizaciones	11
RNF-04	Interoperabilidad con estándares geoespaciales (GIS)	11
RNF-05	Interfaces de línea de comandos (CLI)	12
RI-01	Entidades del estándar GTFS	12
RI-02	Estructuras espaciales derivadas	12
RI-03	Métricas de rendimiento	12

Capítulo 1

Introducción

La gestión y el análisis de datos almacenados en redes geoespaciales se han convertido en parte crucial de múltiples dominios, destacando especialmente en el sector del transporte público. A causa de la creciente complejidad de dichas redes, que combinan información geográfica y temporal, se vuelve necesario el desarrollo de soluciones robustas y eficientes, que a su vez sean capaces de representar de manera conveniente las relaciones entre elementos como paradas, rutas y horarios.

En este contexto, las tecnologías de bases de datos desempeñan un papel crucial. Aunque los sistemas gestores de bases de datos relacionales, es decir, SQL, han sido tradicionalmente la opción dominante por un amplio margen, su fundamento en la **teoría de conjuntos** implica reconstruir las relaciones mediante *joins* durante la ejecución de las consultas.

El surgimiento de los nuevos paradigmas NoSQL, en particular las bases de datos orientadas a grafos, ofrece enfoques alternativos. Se parte de la hipótesis de que estos podrían ofrecer ventajas significativas en eficiencia y expresividad de consultas, al permitir el almacenamiento de los datos cerca de otros con los que están relacionados, independientemente de su tipo, y la recuperación de información mediante la navegación a través de la **topología** de los grafos. Todo ello posibilitaría optimizar, por ejemplo, el uso de la memoria caché.

Este trabajo aborda precisamente el reto de gestionar redes geoespaciales de transporte público, explorando y comparando dos aproximaciones tecnológicas diferentes: un sistema relacional consolidado como PostgreSQL, junto a su extensión geoespacial PostGIS, y la extensión de análisis de redes pgRouting, frente a una de las opciones mejor establecidas en el ámbito de las bases de datos de grafos, Neo4J, utilizando la extensión Neo4J Spatial. En ambos casos se ha optado por tecnologías estables y maduras para garantizar la fiabilidad de los sistemas.

En cuanto a los conjuntos de datos empleados, estos siguen el estándar de facto **GTFS** (General Transit Feed Specification), y representan redes a diferentes

escalas, incluyendo tanto entornos urbanos (Bogotá, Praga, Hong Kong...), como regionales (por ejemplo, Galicia), a fin de poder considerar la escalabilidad que cada sistema ofrece.

Dado que no existe una manera simple de trasladar la información, se busca desarrollar mecanismos para **importar y modelar** las redes en ambos sistemas gestores de bases de datos y, posteriormente, implementar un conjunto representativo de **consultas** pensadas para satisfacer necesidades de información típicas, ya sean del usuario final o de las entidades gestoras del transporte público.

Una vez recuperada la información necesaria, el siguiente paso es presentar los resultados de manera amigable a los usuarios, y, para lograr esto, proporcionar una solución para la **visualización** de los mismos, a través del software QGIS y de la biblioteca Folium, de Python, facilitando su interpretación sobre representaciones cartográficas.

En último lugar, se realiza una **evaluación comparativa** de los dos sistemas, que no se limita a métricas cuantitativas, es decir, el rendimiento, sino que también incluye apartados cualitativos como la facilidad para modelar la red geoespacial y la comodidad a la hora de expresar las consultas en el lenguaje de cada sistema. Con ello, se busca determinar las fortalezas e inconvenientes de cada una de las alternativas y clarificar en qué ocasiones resulta preferible su uso. Estos aspectos se ven reflejados en los resultados presentados al final de la sección.

A lo largo del resto de la memoria se profundiza en diversos aspectos del trabajo. Tras esta introducción, se definen las necesidades de información y funcionalidades relevantes para los usuarios y gestores del sistema de transporte público (capítulo 2), se detalla la solución propuesta, tanto a nivel de modelado como de importación y consulta (capítulo 3), se documenta el plan de pruebas seguido para validar los requisitos previamente planteados (capítulo 4), se realiza una evaluación comparativa tanto del rendimiento como de la facilidad de expresión de consultas en cada uno de los sistemas (capítulo 5) y, finalmente, se presentan las conclusiones obtenidas a raíz de la comparación entre ambos sistemas, así como líneas de trabajo futuras para ampliar el alcance del proyecto (capítulo 6).

Por otra parte, la memoria se complementa con tres anexos: el manual técnico, para las personas que modifiquen el sistema o lleven a cabo algún desarrollo posterior (anexo A), el manual de usuario, que explica cómo instalar, configurar y utilizar el software (anexo B), y la licencia (anexo C).

Capítulo 2

Especificación de requisitos

En este capítulo se proporciona una descripción detallada de los requisitos del sistema, tanto a nivel funcional como no funcional y de información.

2.1. Descripción de dominio y datos

Para que la definición de los requisitos del sistema sea clara, es necesario establecer los conceptos del dominio del transporte público. El sistema se basa en la especificación GTFS[3] (General Transit Feed Specification), estándar de facto en el sector, lanzado originalmente en 2006 por Google.

Estructuralmente, un conjunto de datos GTFS se distribuye como una colección de archivos de texto plano (en formato CSV), donde cada archivo representa una tabla en el modelo relacional (por ejemplo, `stops.txt` o `routes.txt`). Los vínculos entre las distintas entidades se establecen mediante identificadores que actúan como claves primarias y foráneas.

Más allá de su estructura, a nivel semántico, la información describe una red de transporte con dimensiones *espaciales* y *temporales*. Las paradas funcionan como elementos geoespaciales fijos, y las conexiones entre ellas no son estáticas, sino que dependen de los trayectos programados. Esta naturaleza topológica es precisamente lo que motiva la evaluación de las bases de datos orientadas a grafos.

Las entidades más importantes consideradas por este estándar se muestran en la tabla 2.1.

Entidad	Definición
Agencia	Empresa u organización encargada de operar un sistema de transporte público.
Servicio	Disponibilidad temporal de un grupo de trayectos (por ejemplo, "Lunes a Viernes", o "Fines de semana").
Trayecto	Secuencia lineal de paradas consecutivas que se recorren siguiendo un horario determinado.
Ruta	Agrupación lógica de trayectos similares o idénticos visible al pasajero, identificada por un nombre corto (por ejemplo, "Línea 1").
Parada	Ubicación física definida por su latitud y longitud, donde los vehículos se detienen para recoger o dejar pasajeros.
Horario de parada (<i>Stop Time</i>)	Asociación de un trayecto con una parada específica, indicando hora de llegada y salida, así como orden en la secuencia del trayecto.
Forma	Representación geométrica detallada del recorrido físico realizado por el vehículo entre paradas.

Cuadro 2.1: Principales entidades en el estándar GTFS.

2.2. Alcance

El sistema **permitirá**:

- Cargar conjunto de datos que siga el formato GTFS, incluyendo sus apartados opcionales, pero solamente en formato CSV.
- Ejecutar un conjunto predefinido de consultas relevantes sobre los datos cargados, así como consultas de interés para los propios usuarios.
- Visualizar de forma básica la red así como los resultados de ciertas consultas espaciales.
- Comprobar el tiempo de importación de los conjuntos de datos y de ejecución de las consultas.

El sistema **no permitirá**:

- Cargar un conjunto de datos en formato GeoJSON.
- Modificar los datos de manera interactiva una vez cargados.
- Distinguir múltiples usuarios o roles.

2.3. Requisitos funcionales

RF-01: Importación de datos

Descripción: El sistema debe permitir importar un conjunto de datos que siga el estándar GTFS, de acuerdo con su versión de enero de 2025, con todos sus archivos en formato CSV y sin campos faltantes.

Prioridad: Máxima.

Entradas: Nombre del conjunto de datos a importar, ya ubicado en el directorio de datasets del sistema.

Salidas: Mensaje de confirmación o error, informando al usuario del resultado.

Notas: Aunque no pueden faltar campos en los archivos, se admiten valores nulos.

RF-02: Validación de integridad y restricciones

Descripción: El sistema debe asegurar que los datos importados cumplen con las restricciones definidas por el estándar GTFS, lo que incluye verificación de tipos, presencia de campos obligatorios, e integridad referencial entre las entidades.

Prioridad: Alta.

Entradas: Conjunto de datos que está siendo importado.

Salidas: En caso de error, mensaje que señale las restricciones no satisfechas por el conjunto de datos.

Notas: Este proceso no requiere interacción por parte del usuario, sino que se lleva a cabo de forma automática durante la importación.

RF-03: Servicios de transporte activos

Descripción: El sistema debe ser capaz de determinar los servicios de transporte activos para una fecha dada.

Prioridad: Alta.

Entradas: Fecha en la que realizar la búsqueda.

Salidas: Lista de identificadores de servicios activos.

RF-04: Estadísticas diarias

Descripción: El sistema debe ser capaz de calcular el número de trayectos, rutas y paradas activas en la red de transporte para un rango de fechas dado.

Prioridad: Media.

Entradas: Fechas inicial y final del rango.

Salidas: Lista de trayectos, rutas y paradas activas para cada uno de los días del rango.

RF-05: Horas de salida de trayectos

Descripción: El sistema debe ser capaz de encontrar las horas de salida para todos los trayectos vinculados a una ruta, desde una parada concreta y en una fecha especificada.

Prioridad: Alta.

Entradas: Identificadores de ruta y parada, fecha de consulta.

Salidas: Lista de trayectos, junto a su destino, y horas de salida, ordenada temporalmente.

RF-06: Tiempos de espera en la red

Descripción: El sistema debe ser capaz de calcular, para todas las rutas, los tiempos de espera mínimos, promedio y máximos, así como la desviación estándar, agregando estos a lo largo de cada una de las paradas, para una fecha concreta.

Prioridad: Media.

Entradas: Fecha de consulta.

Salidas: Lista de tiempos de espera mínimos, promedio y máximos, junto a su desviación estándar, para cada una de las rutas.

RF-07: Próximas salidas

Descripción: El sistema debe ser capaz de determinar, para una parada concreta, y una fecha y hora dadas, los próximos trayectos que parten desde la misma, ordenados por tiempo de salida.

Prioridad: Alta.

Entradas: Identificador de parada, fecha y hora de consulta.

Salidas: Lista de próximos trayectos, con ruta, destino y hora de salida, ordenados por esta última.

RF-08: Segmentos con solapamiento

Descripción: El sistema debe ser capaz de analizar qué pares de paradas consecutivos son cubiertos por un mayor número de rutas.

Prioridad: Media.

Entradas: —

Salidas: Lista de pares de paradas, número y nombre de rutas que los cubren, ordenada por el número de rutas de manera descendente.

Notas: El interés de esta consulta reside en que permite determinar planificación redundante en la red de transporte.

RF-09: Ranking de rutas por relevancia

Descripción: El sistema debe ser capaz de calcular, dadas una fecha y hora, las rutas más importantes en base al número de trayectos activos y la frecuencia de estos.

Prioridad: Baja.

Entradas: Fecha y hora de consulta.

Salidas: Lista de rutas ordenada de manera descendente por número de trayectos y de manera ascendente por la frecuencia de los mismos.

RF-10: Ranking de rutas por velocidad

Descripción: El sistema debe ser capaz de encontrar las rutas más rápidas de la red, considerando la agregación de la velocidad promedio de cada uno de sus trayectos.

Prioridad: Media.

Entradas: —

Salidas: Lista de rutas con velocidad promedio y número de trayectos agregados para el cálculo, ordenada de manera descendente por dicha velocidad.

RF-11: Paradas cercanas

Descripción: El sistema debe ser capaz de hallar las paradas más cercanas a una ubicación proporcionada por el usuario, dada una distancia de búsqueda.

Prioridad: Alta.

Entradas: Latitud y longitud de la localización y radio de búsqueda.

Salidas: Lista de paradas próximas, junto a sus latitudes, longitudes y distancias respecto al punto dado, ordenada por estas últimas.

RF-12: Paradas más transitadas

Descripción: El sistema debe ser capaz de determinar las paradas más importantes en la red de transportes para una fecha dada, de acuerdo con el número de trayectos que parten de ellas.

Prioridad: Baja.

Entradas: Fecha de consulta.

Salidas: Lista de paradas con el número y nombre de rutas que las atraviesan, el número de trayectos que parten de ellas y la hora de la primera y última salidas, ordenada por el número de trayectos de manera descendente.

RF-13: Histograma de comienzo de trayectos

Descripción: El sistema debe ser capaz de generar, dada una longitud de intervalo y una fecha, un histograma que muestre cuántos trayectos comienzan en cada intervalo temporal, a lo largo del día.

Prioridad: Alta.

Entradas: Longitud de intervalo en minutos y fecha de consulta.

Salidas: Lista de intervalos en que se divide la jornada y número de trayectos que inician en cada uno de ellos, ordenada de manera temporal.

RF-14: Mapa de calor de paradas

Descripción: El sistema debe ser capaz de dividir de forma geométrica la región sobre la que se encuentra la red de transportes y, para cada una de esas subdivisiones, determinar el número de paradas contenidas.

Prioridad: Baja.

Entradas: Tamaño de las subdivisiones.

Salidas: Lista de geometrías (cada una de ellas representando una subdivisión) y número de paradas que se encuentran en cada una.

Notas: La implementación de este requisito está supeditada al soporte proporcionado por las bibliotecas PostGIS y Neo4J Spatial. En ningún caso se implementará el algoritmo de segmentación de manera manual.

RF-15: Análisis de alcanzabilidad

Descripción: El sistema debe ser capaz de encontrar, a partir de una parada inicial, la hora mínima de llegada para cada una de las otras paradas del sistema, considerando no solo los trayectos de la red de transporte, sino también los transbordos entre estaciones y la posibilidad de caminar entre paradas.

Prioridad: Alta.

Entradas: Identificador de la parada inicial, fecha y hora de partida.

Salidas: Lista de paradas de la red, hora mínima de llegada a cada una de ellas, parada previa en el viaje, y acción realizada desde la parada previa (trayecto, transbordo o caminar).

Notas: Dado que el resultado varía en función tanto de elementos dependientes del tiempo (trayectos) como independientes (transbordos, desplazamiento a pie), es probable que el soporte de bibliotecas no resulte suficiente y sea necesario implementar un algoritmo propio.

RF-16: Camino más corto

Descripción: El sistema debe ser capaz de encontrar, a partir de una parada de partida, el camino que le permite llegar a otra de destino de la manera más rápida posible, considerando una fecha y hora de inicio concretas.

Prioridad: Alta.

Entradas: Identificador de las paradas de partida y destino, fecha y hora de inicio.

Salidas: Lista de paradas a recorrer, así como la hora de llegada a cada una de ellas y la acción tomada en la parada previa (trayecto, transbordo o caminar).

RF-17: Visualización espacial de resultados

Descripción: El sistema debe permitir al usuario generar mapas que representen los resultados espaciales producidos por la base de datos (por ejemplo, un mapa de calor sobre la densidad de paradas en que, al colocar el ratón sobre una subdivisión del mapa, se muestre el número concreto).

Prioridad: Media.

Entradas: Datos requeridos por la consulta cuyo resultado se quiera visualizar.

Salidas: Mapa interactivo que muestre el resultado de la consulta.

RF-18: Generación de gráficas

Descripción: El sistema debe permitir al usuario, en aquellos casos en que la consulta sea de índole temporal, generar una representación visual no interactiva (por ejemplo, una gráfica de mancuerna para los tiempos de espera en cada ruta).

Prioridad: Alta.

Entradas: Datos requeridos por la consulta cuyo resultado se quiera visualizar.

Salidas: Gráfica que muestre el resultado de la consulta.

RF-19: Medición de rendimiento

Descripción: El sistema debe permitir al usuario medir el tiempo de ejecución tanto del proceso de importación como de las consultas predefinidas.

Prioridad: Media.

Entradas: Conjunto(s) de datos a importar para evaluar el rendimiento de la importación, o consulta(s) a ejecutar en caso contrario.

Salidas: Lista de tiempos de ejecución para la acción realizada.

2.4. Requisitos no funcionales

RNF-01: Escalabilidad respecto al conjunto de datos

Descripción: El sistema debe poder importar conjuntos de datos GTFS de hasta 500 MB de tamaño en texto plano, utilizando hardware no especializado (ordenadores personales, con 16 GB de RAM) sin incurrir en errores de desbordamiento de memoria (OOM).

Prioridad: Alta.

Criterio de verificación: Importación de un conjunto de datos GTFS de aproximadamente 500 MB.

RNF-02: Extensibilidad del catálogo de consultas

Descripción: El sistema debe permitir añadir nuevas consultas predefinidas introduciendo únicamente el código SQL o Cypher de dichas consultas en los ficheros correspondientes, sin necesidad de refactorizar el código.

Prioridad: Alta.

Criterio de verificación: Adición de una nueva consulta introduciendo únicamente su declaración (nombre y argumentos) y definición (algoritmo), sin modificar o borrar código previamente existente.

RNF-03: Portabilidad de las visualizaciones

Descripción: El sistema debe garantizar que las visualizaciones generadas utilizan formatos portables (PNG en el caso de las gráficas, HTML en el de los mapas interactivos).

Prioridad: Media.

Criterio de verificación: Apertura de archivos generados en aplicaciones de escritorio estándar (visor de imágenes y navegador).

RNF-04: Interoperabilidad con estándares geoespaciales (GIS)

Descripción: El sistema debe almacenar la información geoespacial de acuerdo con los estándares definidos por el Open Geospatial Consortium, como, por ejemplo, utilizando los tipos WKB y WKT.

Prioridad: Alta.

Criterio de verificación: Conexión desde una herramienta GIS externa como QGIS, realización de una consulta de prueba y procesamiento por parte de la herramienta de los datos devueltos por la BBDD.

RNF-05: Interfaces de línea de comandos (CLI)

Descripción: El sistema debe exponer su funcionalidad a través de herramientas (scripts) ejecutados en la línea de comandos, tanto para el proceso de importación como para la generación de visualizaciones, tomando como argumentos las entradas necesarias (nombre del dataset o argumentos de la consulta).

Prioridad: Media.

Criterio de verificación: Ejecución en un terminal de los scripts de importación y producción de visualizaciones.

2.5. Requisitos de información

RI-01: Entidades del estándar GTFS

Descripción: El sistema debe almacenar y gestionar toda la información definida en la especificación GTFS (Static), incluyendo, entre otros, agencias y servicios de transporte, paradas, rutas, trayectos, tarifas y datos de accesibilidad, preservando el componente semántico de dicha información, pero no necesariamente la representación.

RI-02: Estructuras espaciales derivadas

Descripción: El sistema debe generar y mantener representaciones enriquecidas de los datos originales para su análisis, y, en particular, objetos geométricos sobre la superficie terrestre (puntos asociados a la ubicación de las paradas, líneas que describan el trazado de cada trayecto, y polígonos que delimiten las zonas de tarificación consideradas por los servicios de transporte).

RI-03: Métricas de rendimiento

Descripción: El sistema debe almacenar los resultados cuantitativos de las pruebas de rendimiento, siendo estos fundamentalmente los tiempos de importación de los conjuntos de datos y de ejecución de las consultas.

Capítulo 3

Diseño

Una vez definido el sistema a construir, este capítulo aborda el diseño del mismo, haciendo uso de una aproximación *top-down*, es decir, proporcionando en primer lugar una visión global de la arquitectura y profundizando progresivamente en los detalles de implementación.

3.1. Arquitectura del sistema

En términos generales, el sistema está diseñado siguiendo una **arquitectura modular** y desacoplada, donde un controlador orquesta el flujo de datos y el proceso de pruebas. Adicionalmente, y como se muestra en la figura 3.1, el sistema está compuesto por tres capas:

1. **Control:** Incluye el ya mencionado controlador, que procesa los conjuntos de datos antes de su importación, coordina las transacciones en las bases de datos y gestiona la ejecución de las pruebas. Para la comunicación, utiliza `psycopg` y `neo4j-driver` sobre TCP/IP.
2. **Persistencia:** Compuesta por los motores de base de datos, PostgreSQL[5] y Neo4J[7], que se despliegan como contenedores Docker aislados, y operan de manera independiente entre sí.
3. **Presentación:** Abarca tanto la integración con herramientas externas para la visualización (QGIS), como los artefactos generados para su análisis, en forma de mapas interactivos o gráficas.

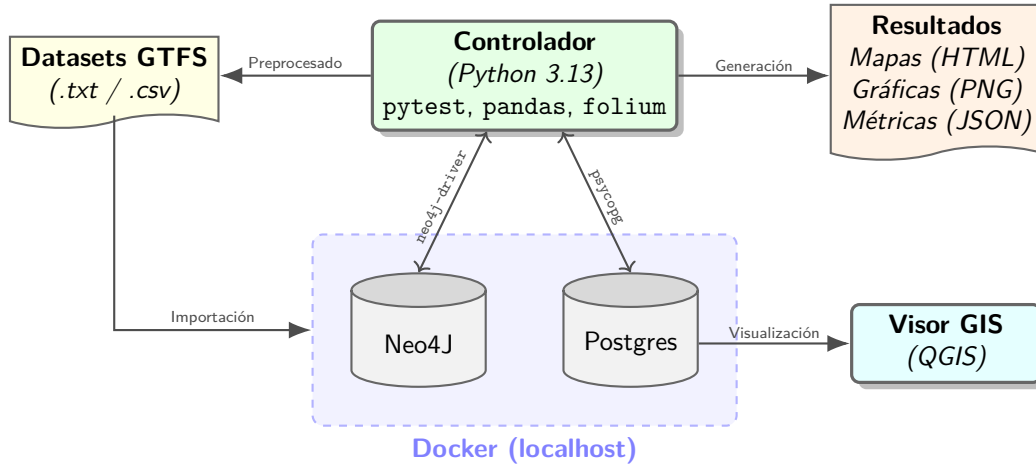


Figura 3.1: Diagrama de la arquitectura general del sistema.

3.2. Infraestructura y despliegue

3.2.1. Aislamiento y reproducibilidad

Para garantizar que los componentes de la capa de persistencia, es decir, los motores de base de datos, no interactúen entre sí, generando incompatibilidades, estos se virtualizan mediante contenedores Docker[11]. Además, de esta forma se logra la reproducibilidad e independencia del sistema operativo y bibliotecas subyacentes, al encapsular las dependencias de cada motor y sus extensiones geoespaciales.

Las imágenes Docker seleccionadas son `pgrouting:17-3.5-main`, en el caso de PostgreSQL, que incluye tanto la extensión `postgis`, como `pgrouting`, y en el de Neo4J, `neo4j:5.19.0-enterprise`, en lugar de la versión Community, para poder hacer uso de, entre otros, restricciones de tipos y existencia de propiedades.

No obstante, a diferencia de las arquitecturas de microservicios convencionales, no se considera necesario hacer uso de Docker Compose, optando por scripts de lanzamiento de cada base de datos de manera independiente, dado que los motores no necesitan comunicarse entre sí (*shared-nothing architecture*), e introducir un fichero `docker-compose.yml` no aportaría ningún beneficio, y forzaría al usuario a lanzar ambos motores incluso aunque solo vaya a utilizar uno, desperdiciando recursos del sistema.

3.2.2. Conexiones

Una de las responsabilidades principales del controlador Python[12] es gestionar el ciclo de vida de las conexiones, durante tanto el proceso de prueba como la explotación del sistema, mediante el uso de *fixtures* proporcionadas por *pytest*.

Para ello, se sigue el flujo descrito en la figura 3.2, lo que permite garantizar al mismo tiempo que el sistema no desperdicia recursos abriendo y cerrando más conexiones de las necesarias y que, cuando termina la ejecución de consultas o *statements*, los sockets son liberados correctamente, sin generar *leaks*.

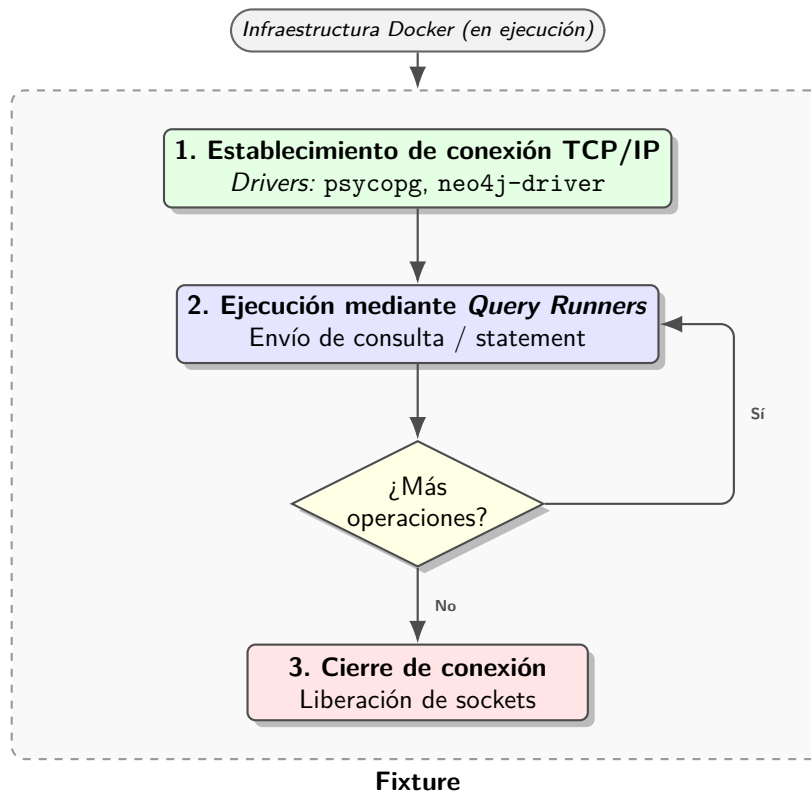


Figura 3.2: Ciclo de vida de las conexiones del sistema.

Cabe mencionar que el flujo presentado se aplica únicamente al controlador Python y, si se opta por utilizar un software GIS de escritorio, como QGIS, este se encarga de establecer y administrar la conexión con PostgreSQL, de forma independiente, y a partir ciertos parámetros proporcionados, como el *endpoint* y las credenciales de acceso.

3.2.3. Entorno de ejecución

Hardware

Para asegurar que las métricas de rendimiento del sistema reflejan las capacidades de los motores de bases de datos y no las limitaciones del hardware subyacente, se han establecido los siguientes requisitos mínimos:

- **Procesador:** Intel i7 de octava generación, o su equivalente en AMD (arquitectura x86-64), con soporte para virtualización, necesaria para la ejecución de los contenedores Docker.
- **RAM:** 16 GB, dado que tanto la importación como las consultas cargan una gran cantidad de datos en memoria, especialmente en Neo4J, al almacenar la topología de la red, y se busca evitar el *swapping*.
- **Disco duro:** Un disco de estado sólido (SSD) con al menos 10 GB libres, dado que el proceso de importación es intensivo en E/S, por lo que un disco duro mecánico (HDD) se convertiría en un cuello de botella.

Por otra parte, para la obtención de los resultados presentados en el capítulo 5, los requisitos previamente listados se materializan en un sistema con un procesador Intel i7 10750H, 32 GB de RAM y un SSD con interfaz NVMe.

Software

En cuanto al entorno software, las dependencias principales son:

- **Sistema operativo:** Cualquier distribución de Linux, para poder ejecutar Docker de manera nativa.
- **Python y bibliotecas:** Versión 3.13 del lenguaje, junto con las bibliotecas listadas en el fichero `requirements.txt` del directorio del proyecto: entre otras, `pytest`[13] para la automatización de las pruebas, `folium`[15] para la generación de mapas interactivos y `pandas`[16] para el preprocesado de los conjuntos de datos, así como los drivers de PostgreSQL y Neo4J.
- **Visor GIS:** Cualquier visor GIS que soporte conexiones con PostgreSQL. Se recomienda QGIS[17] dado que es el estándar de la industria.

Para más información sobre las versiones utilizadas, véase el cuadro B.1 del manual de usuario, y el fichero `requirements.txt` del directorio del proyecto.

3.3. Principios y patrones de diseño

La construcción del sistema no se limita exclusivamente a la codificación, sino que también incorpora principios arquitectónicos y patrones de diseño, a fin de garantizar la modularidad, la integridad de los datos y también el mantenimiento de los componentes software.

En este apartado, se presentan algunos de los principios y patrones aplicados, partiendo de la arquitectura y concretando progresivamente en aspectos del código.

3.3.1. Separación de responsabilidades

La definición del modelo de datos se encuentra desacoplada de la implementación de consultas, y estas, a su vez, de la orquestación del sistema. Gracias a ello, la modificación de alguno de los componentes no afecta al resto.

3.3.2. KISS (*Keep it simple, stupid*)

Se opta por utilizar las bibliotecas `psycpg` y `neo4j-driver` de manera directa, en lugar de escoger un ORM pesado como SQLAlchemy, lo que reduce el consumo de recursos sin un impacto tangible en la funcionalidad.

Por otra parte, dado que uno de los objetivos de este trabajo es llevar a cabo una comparativa de rendimiento, esto también garantiza la **transparencia** de las mediciones, asegurando que se observa el rendimiento ofrecido por los motores de base de datos y no el que proporciona el framework escogido.

3.3.3. DRY (*Don't repeat yourself*)

En la implementación de las pruebas, el fichero `confest.py` se encarga de gestionar los detalles comunes, como el flujo de ejecución de los casos de prueba, análogo para todos ellos, evitando la repetición de código y favoreciendo la mantenibilidad.

3.3.4. Defensa en profundidad

Para evitar que el sistema opere sobre datos semánticamente incorrectos, se hace uso de un ***pipeline* de defensa en profundidad**, o pipeline de validación, tal y como se muestra en la figura 3.3. Este pipeline cuenta con varios niveles que garantizan que la información sea coherente:

1. **Preprocesado en Python:** Antes de llevar a cabo la importación, el script `process_dataset.py` comprueba que los ficheros de texto siguen el formato esperado por el sistema, aceptando valores nulos pero no columnas faltantes en los CSV. En caso contrario, se encarga de adaptar dichos ficheros.
2. **Semántica en BBDD:** Durante la carga de datos, y mediante restricciones nativas a los motores de base de datos, (`CHECK`, `NOT NULL`, *triggers*, limitaciones de tipos y claves foráneas), se verifica que los datos se encuentren en los rangos de valores aceptados por el estándar GTFS, y que se almacenen en un formato estandarizado (por ejemplo, todos los colores son cadenas de 6 dígitos hexadecimales, en mayúsculas).
3. **Programación defensiva en queries:** A la hora de recuperar la información, aunque se cuenta con dos capas de defensa previas, las consultas no asumen que los datos son semánticamente correctos, incluyendo comprobaciones adicionales (por ejemplo, cláusulas `WHERE` que garanticen que no se está operando sobre valores nulos).

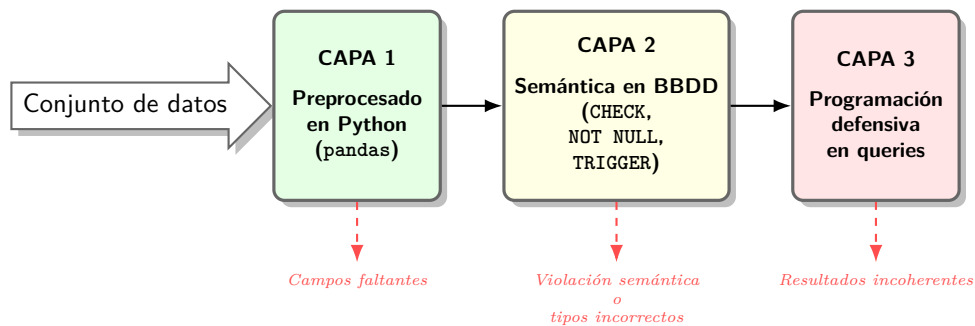


Figura 3.3: *Pipeline* de defensa en profundidad.

3.3.5. Inyección de dependencias

Todo el proceso de pruebas hace uso de *fixtures*, de tal modo que los casos de prueba no crean los objetos que necesitan para su ejecución, sino que es el propio `pytest` el encargado de instanciarlos y pasarlos como argumentos.

3.3.6. Fachada

El controlador se encarga de encapsular la complejidad inherente a la gestión de conexiones en `database.py`, proporcionando al resto de scripts una interfaz simplificada para lanzar sus consultas, mediante los objetos `pg_query_runner` y `neo4j_query_runner`.

3.4. Datos

El diseño de la capa de persistencia se caracteriza por la dualidad de modelos. Aunque la fuente de datos es la misma (los ficheros CSV del estándar GTFS), la aproximación al almacenamiento difiere entre el motor relacional y el orientado a grafos.

3.4.1. Ciclo de vida de la información

A nivel de flujo, la información sigue un modelo lineal caracterizado por su simplicidad:

1. **Preprocesado:** El script `process_dataset.py` se encarga de limpiar el conjunto de datos, garantizando que no hay columnas faltantes (aunque se admiten valores nulos).
2. **Modelado e importación:** Bajo la orquestación de `import.py`, los motores de bases de datos generan el esquema, leen los ficheros CSV del conjunto de datos y almacenan la información del modo descrito en los siguientes subapartados.
3. **Enriquecimiento:** A continuación, tanto PostgreSQL como Neo4J enriquecen la información importada con datos de índole geoespacial, como puntos que representan la ubicación de las paradas o líneas que siguen el trazado de los trayectos.
4. **Recuperación:** Mediante los scripts de consulta o evaluación de rendimiento, ubicados en el directorio `Scripts`, se recupera la información de las bases de datos y se utiliza para producir resultados.
5. **Almacenamiento y visualización:** Los resultados generados son visualizados, en el caso de los mapas o las gráficas, o almacenados en ficheros JSON, en el caso de los tiempos de importación y ejecución.

3.4.2. Modelo relacional (PostgreSQL)

El modelado en PostgreSQL consiste en un *mapeo* casi¹ inmediato (1:1) de las entidades definidas en el estándar GTFS. En la figura 3.4 se puede apreciar

¹Algunos elementos, como las claves primarias con valores potencialmente nulos, no se pueden implementar de forma directa en PostgreSQL, así que se han buscado alternativas: en ese caso, índices que garantizan unicidad.

una simplificación del modelo entidad-relación (MER) seguido, considerando las entidades básicas.

A nivel de implementación, se utiliza un script por cada una de las tablas SQL, ubicados todos ellos en el directorio `Scripts/PostgreSQL/Import`. También, en lugar de almacenar los datos de tipos enumerados (por ejemplo, estatus de accesibilidad de paradas o direcciones de viaje) como valores enteros, se han generado tablas adicionales para cada uno de ellos.

En lo relativo a los componentes opcionales del estándar GTFS, sus tablas se crean siempre, independientemente de que el conjunto de datos vaya a insertar datos en ellas o no.

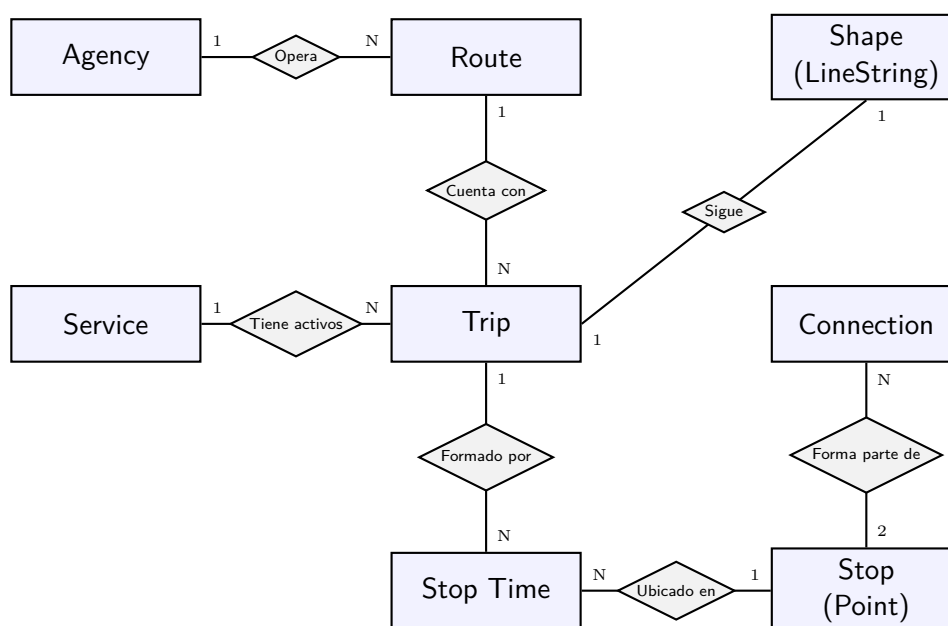


Figura 3.4: Modelo entidad-relación (simplificado) seguido en PostgreSQL.

3.4.3. Modelo orientado a grafos (Neo4J)

En Neo4J, el almacenamiento de la información abandona la estructura tabular para adoptar una topología de red. La transformación se lleva a cabo de la forma más intuitiva: cada clave foránea en el modelo original se transforma en una arista, es decir, una relación explícita, en el grafo. De esta forma, se obtiene una estructura como la representada en la figura 3.5, un meta-grafo simplificado.

Existen otras posibilidades de modelado, siendo una de ellas la representación de los trayectos como múltiples relaciones entre paradas en lugar de nodos independientes. Sin embargo, dicho enfoque presenta múltiples problemas:

- No es posible representar una relación que tenga a otra relación como uno de los extremos, por lo que sería imposible vincular múltiples trayectos a una ruta, y, posteriormente, realizar agregación sobre todos ellos. La única alternativa sería introducir **enormes redundancias** al almacenar toda la información de la ruta en cada relación de cada trayecto.
- A primera vista, parecería muy sencillo identificar la primera parada de un trayecto (la única con una relación de salida y ninguna de entrada), y la última. También resultaría fácil atravesar todas las paradas del trayecto, mediante la cadena de relaciones. No obstante, esto no funciona en realidad a causa de la existencia de **trayectos circulares**, lo cual obligaría a considerar el número de secuencia de cada parada como en el diseño actual.
- Por otra parte, para recorrer un trayecto, sería necesario filtrar las relaciones que salen de cada una de las paradas para encontrar las que corresponden al trayecto en cuestión. El coste de esta operación se puede reducir con índices, pero Neo4J está más optimizado para atravesar caminos que filtrar grandes cantidades de relaciones. En consecuencia, se piensa que esta alternativa ofrecería un **rendimiento igual o peor**.

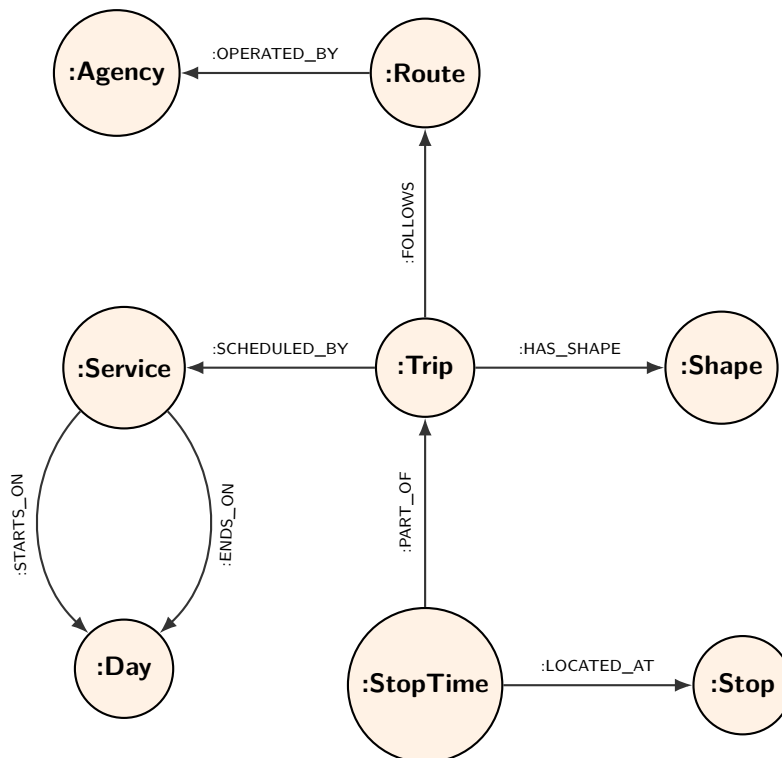


Figura 3.5: Meta-grafo (simplificado) seguido en Neo4J.

3.5. Algoritmos y consultas

El sistema implementa dos clases de lógica de procesamiento: mientras que la mayoría de consultas son declarativas, y su ejecución es planificada por los motores de base de datos, los algoritmos de alcanzabilidad y búsqueda de caminos más cortos son imperativos, y se implementan como procedimientos.

A causa de ello, y dado que Neo4J no proporciona soporte nativo para código imperativo, dichas consultas no se han implementado en el motor. Sería necesario desarrollar un plugin en Java que interopere con Neo4J, pero esto se considera fuera del ámbito del proyecto.

3.5.1. Lógica declarativa

La amplia mayoría de las consultas predefinidas han sido implementadas de modo puramente declarativo, tanto en PostgreSQL como en Neo4J. En esta sección se presentan algunos de los detalles de implementación más destacables.

Procesamiento de secuencias

Una parte fundamental del análisis de datos de transporte requiere operar sobre secuencias ordenadas de datos. Algunos ejemplos de ello son el cálculo de duraciones en los trayectos, la determinación de tiempos de espera en las paradas, o la identificación de segmentos de rutas.

Para satisfacer esta necesidad, se han utilizado dos patrones de consulta, uno por cada motor de base de datos: **funciones de ventana** en PostgreSQL y **procesamiento de colecciones** en Neo4J.

En el primer caso, PostgreSQL permite evitar los costosos *self-joins* mediante el uso de funciones de ventana, como **LEAD**, que permite acceder a valores posteriores en una secuencia ordenada. Además, también se puede acceder a los extremos de la ventana mediante **FIRST_VALUE** y **LAST_VALUE**. En la figura 3.6 se muestra un ejemplo de su uso para calcular las duraciones de los trayectos.

Por otro lado, en el caso de Neo4J, Cypher[8] no dispone de un equivalente directo a las funciones de ventana, por lo que se opta por la materialización de secuencias en listas, mediante **collect**, y su procesamiento con funciones proporcionadas por la biblioteca APOC[9], o con extensiones de lista (*list comprehensions*). En la figura 3.7 se muestra un ejemplo de su uso para calcular todos los tiempos de espera entre trayectos para cada par (ruta, parada).

```

SELECT DISTINCT
  trip_id,
  EXTRACT(
    EPOCH FROM (
      LAST_VALUE(arrival_time::interval) OVER trip_window -
      FIRST_VALUE(departure_time::interval) OVER trip_window
    )
  ) AS duration_seconds
FROM stop_time
WINDOW trip_window AS (
  PARTITION BY trip_id
  ORDER BY stop_sequence
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
)

```

Figura 3.6: Ejemplo de funciones ventana en PostgreSQL.

```

MATCH (r:Route)<-[:FOLLOWS]-(t:Trip)-[:SCHEDULED_BY]->(:Service {id: s_id})
MATCH (s:Stop)<-[:LOCATED_AT]-(stt:StopTime)-[:PART_OF]->(t)

[...]

WITH r, s, direction_id as direction_id, collect(departure_time) as dt
WHERE size(dt) >= 2      // A single departure is not enough.

WITH r, s, direction_id,
  [i IN range(0, size(dt) - 2) | dt[i+1] - dt[i]] as rs_headways

```

Figura 3.7: Ejemplo de procesamiento de colecciones en Neo4J.

Agregación en conjuntos

Algunas consultas, como `daily_status`, pueden ofrecer un rendimiento muy superior si agrupamos todos los días con los mismos servicios activos en un solo conjunto, dado que se eliminan los cálculos duplicados.

En PostgreSQL, esto se logra mediante la creación de una tabla adicional, llamada `day_service_sets`, cuyas filas contienen tuplas (`service_dates`, `service_set`), donde el primer elemento es un conjunto de fechas con los mismos servicios activos, y el segundo, los identificadores de dichos servicios.

Asimismo, para garantizar la eficiencia a la hora de consultar los datos de esta tabla, se crean **índices GIN**, utilizados junto a los operadores `&&` y `<@` en la implementación de las consultas.

Optimización espacial

Varias de las consultas predefinidas hacen uso de datos espaciales. Por ejemplo, en `stops_within_distance`, es necesario determinar puntos geoméricamente cercanos.

El enfoque *naïve* consiste en comparar la distancia con todo el resto de puntos, pero esto resulta ser muy costoso en términos de rendimiento.

Afortunadamente, tanto PostGIS[6] como Neo4J-Spatial[10] proporcionan mecanismos para acelerar estas consultas, como los **índices GiST**, de los que se ha hecho uso, creándolos inmediatamente después de llevar a cabo la importación de los datos.

Metaconsultas

Una de las mayores limitaciones de Neo4J es su incapacidad de definir funciones procedimientos mediante Cypher, requiriendo hacer uso de plugins externos escritos en Java.

Sin embargo, y dado que se busca proporcionar un catálogo de consultas, este problema se ha abordado utilizando un patrón de **metaconsultas**. Gracias a la biblioteca **APOC**, podemos ejecutar código Cypher dinámico, con el procedimiento `apoc.cypher.run`, lo que a su vez nos permite almacenar el texto de las consultas como una propiedad en un nodo etiquetado como *CypherQuery*, recuperarlo posteriormente, y lanzar dicha consulta.

En la figura 3.8 se puede apreciar la invocación de una de las consultas predefinidas a través de esta solución.

```
MATCH (cq: CypherQuery {name: 'headway_stats'})
CALL apoc.cypher.run(cq.statement, {curr_date: '2025-12-25'})
YIELD value
RETURN value;
```

Figura 3.8: Código de invocación de una consulta predefinida en Neo4J.

3.5.2. Enrutamiento híbrido

El componente más complejo del sistema a nivel algorítmico es la función de cálculo de alcanzabilidad, denominada `earliest_arrivals`, que se utiliza también para hallar el camino más corto entre dos paradas.

Dada la naturaleza del problema, que requiere considerar tanto factores dependientes del tiempo (los trayectos) como independientes (los transbordos y los desplazamientos a pie), no es posible hacer uso ni del algoritmo de Dijkstra[1], ni del *Connection Scan Algorithm*[2] (CSA). Asimismo, tampoco se puede optar por un algoritmo proporcionado por *pgrouting* o Neo4J, ya que no soportan factores dependientes del tiempo.

Para ilustrarlo, consideremos dos paradas, p_1 , p_2 . Desplazarse entre ellas es posible tanto a pie, tardando 10 minutos, como mediante un trayecto, que dura 5 minutos y sale a las 9:03. Si utilizamos el algoritmo de Dijkstra, el coste de las aristas *cambiaría constantemente*, siendo de 8 minutos a las 9:00 (3 de espera + 5 de viaje), 5 a las 9:03, y 10 a las 9:04 (el trayecto ya no está disponible), por lo que no es posible construir el grafo de conexiones sobre el que aplicarlo. Por otra parte, el algoritmo CSA está basado en iterar sobre un vector de conexiones ordenadas por tiempo, de modo que la conexión a pie no se puede representar al no tener un tiempo de salida fijo.

Por tanto, como parte del proyecto, se desarrolla e implementa un algoritmo híbrido, que toma el componente fundamental de CSA, iterando sobre las conexiones entre nodos ordenadas temporalmente por hora de salida, y añade una fase de relajación local, similar al algoritmo de Dijkstra.

Fundamento teórico

El algoritmo CSA original mantiene un vector de estado E , donde $E[p]_t$ representa la hora mínima de llegada a la parada p en el instante t . Durante la ejecución del algoritmo, este vector se actualiza a medida que avanza el tiempo. Dado que el coste de desplazamiento nunca es negativo, se puede asegurar que:

$$\forall p, t, \quad E[p]_t \leq t \implies \nexists t' > t : E[p]_{t'} < E[p]_t \quad (\text{Optimalidad global})$$

A modo de ejemplo, supongamos que, en un cierto paso de la ejecución del algoritmo, la hora más temprana de llegada a una parada p son las 12:30. Además, la siguiente conexión que vamos a procesar tiene también como hora de salida las 12:30. En este caso, podemos asegurar que la hora de llegada a p no puede mejorar, ya que cada conexión tiene un coste temporal (y se avanza sobre ellas en orden cronológico), es decir, **no se puede viajar al pasado**.

En la variante híbrida, al confirmar un tiempo de llegada globalmente óptimo t , a una parada p , se inyectan conexiones de coste estático y tiempo de salida t (representando caminar y cada uno de los posibles transbordos), y se procesan inmediatamente para encontrar tiempos de llegada a los vecinos de p potencialmente mejores a los almacenados en el vector de estado E .

Por otra parte, como hemos mencionado antes, CSA proporciona resultados

óptimos únicamente si garantizamos que $E[p]_t$ representa la hora mínima de llegada a la parada p en el instante t . Dado que las relajaciones de los nodos pueden mejorar el tiempo de llegada a alguno de sus vecinos, es imprescindible que dichas relajaciones se produzcan en cuanto se confirme la optimalidad global de un camino.

Utilizando el ejemplo anterior, si la hora de llegada óptima a p son las 12:30, **antes** de procesar la conexión con esa misma hora de salida, se buscan desplazamientos *a pie* desde p a sus paradas vecinas, que puedan mejorar las horas de llegada encontradas hasta ese punto, para mantener el invariante en que se basa el algoritmo CSA.

Implementación

En la figura 3.9 se muestra una implementación simplificada del algoritmo en PL/pgSQL. Este está formado por las siguientes etapas:

1. **Inicialización (líneas 1-10):** Se preparan las estructuras de datos necesarias, es decir, la tabla de resultados, la cola de prioridad, que almacena las paradas pendientes junto a su tiempo de llegada *tentativo*, y las listas de vecinos y transferencias de cada parada.
2. **Iteración cronológica (líneas 12-17):** Se avanza sobre la lista de conexiones (segmentos de trayectos) ordenada cronológicamente por tiempo de salida, del mismo modo que en el CSA original.
3. **Confirmación de resultados óptimos (líneas 18-24):** Cuando se encuentran tiempos de llegada globalmente óptimos, se eliminan de la cola de prioridad las paradas asociadas, y se itera sobre estas.
4. **Expansión a paradas vecinas (líneas 25-45):** Partiendo de las paradas cuyo camino óptimo se ha encontrado, se busca mejorar el tiempo de llegada a sus vecinos, ya sea andando o mediante transferencias.
5. **Procesamiento de conexión (líneas 47-52):** Se procesa la conexión mediante vehículo actual, comprobando si mejora el tiempo de llegada a su destino.

```

1  -- Initialize data structures.
2  -- Array of results that is progressively filled.
3  SELECT ... INTO results FROM "stop" s;
4
5  -- Priority queue of stops, sorted by earliest arrival time.
6  CREATE TEMP TABLE pqueue (...) ON COMMIT DROP;
7  INSERT INTO pqueue SELECT ... FROM "stop" s;
8
9  -- Lists of neighbors and transfers for each stop.
10 neighbors := (SELECT ...); transfers := (SELECT ...);
11
12 -- Loop through chronologically sorted connections.
13 FOR conn IN
14     SELECT *
15     FROM connections c
16     ORDER BY departure_time ASC
17 LOOP
18     -- Iterate over arrival times confirmed as optimal.
19     IF last_scanned_departure_time < conn.departure_time THEN
20         FOR conf_stop IN
21             -- Remove the stops from the queue after processing them.
22             DELETE FROM pqueue pq
23             WHERE pq.arrival_time <= conn.departure_time
24             RETURNING *
25         LOOP
26             -- Iterate over neighbors.
27             FOREACH nei IN ARRAY neighbors[conf_stop.stop_idx].val
28             LOOP
29                 -- Calculate arrival time at the neighbor stop.
30                 new_arrival := ...
31
32                 -- Update arrival time if earlier.
33                 IF new_arrival < results[nei].earliest_arrival THEN
34                     results[nei] = ...;
35                     UPDATE pqueue SET ... WHERE ...;
36                 END IF;
37             END LOOP;
38
39             -- Iterate over transfers.
40             FOREACH transfer IN ARRAY transfers[conf_stop.stop_idx].val
41             LOOP
42                 -- Analogous to neighbor handling.
43             END LOOP;
44         END LOOP;
45     END IF;
46
47     -- Process the next regular connection (trip).
48     -- We check if it's reachable from its departure stop.
49     IF results[departure_stop].earliest_arrival <= conn.departure_time THEN
50         -- Analogous to the neighbor handling.
51     END IF;
52 END LOOP;

```

Figura 3.9: Implementación simplificada del algoritmo CSA híbrido.

Capítulo 4

Pruebas

Una vez construido el sistema, se procede a evaluar su correctitud mediante la ejecución de un plan de pruebas, que sirve también de base para comparativas entre motores, ya en el capítulo 5.

4.1. Estrategia de pruebas

4.1.1. Alcance

El plan de pruebas busca asegurar el correcto funcionamiento de los siguientes aspectos:

- **Integración de componentes:** Conectividad e interoperabilidad entre los sistemas gestores de bases de datos y el módulo de control, en Python, a través de drivers. Su verificación se lleva a cabo de manera implícita al ejecutar las demás pruebas.
- **Integridad de datos:** Preservación de la información contenida en los conjuntos de datos durante el proceso de importación, incluyendo también la integridad referencial y respetando los tipos de datos.
- **Algoritmos de consulta:** Exactitud de los resultados devueltos por las consultas implementadas. Dado que se utilizan dos sistemas gestores de bases de datos diferentes, el objetivo principal es asegurar que los resultados proporcionados por ambos son iguales, lo que sirve de evidencia sobre su equivalencia semántica.

Cabe mencionar que este capítulo se centra exclusivamente en la **corrección**, es decir, que el sistema funcione de la manera esperada, y no en el rendimiento.

4.1.2. Herramientas e infraestructura

La tecnología principal sobre la que se articula el plan de pruebas es **pytest**. Esta elección responde a su capacidad de llevar a cabo los siguientes aspectos:

- Automatización de la ejecución, permitiendo lanzar todas las pruebas con un solo comando.
- Reporte detallado de errores, indicando exactamente qué casos de prueba han fallado.
- Creación de un entorno controlado que garantice la reproducibilidad de los resultados.
- Gestión de elementos comunes a todas las pruebas, como las conexiones a ambas bases de datos, haciendo uso de *fixtures*.

En cuanto a otras bibliotecas de Python, se emplean **psycopg** y **neo4j-driver** para las conexiones con las bases de datos, y **hypothesis** para las pruebas basadas en propiedades.

Al igual que durante la operación habitual del sistema, las bases de datos se ejecutan como contenedores de **Docker**, lo que garantiza aislamiento del sistema operativo y de sus bibliotecas.

4.1.3. Conjuntos de datos

A fin de comprobar la robustez del sistema frente a diferentes estructuras de red, convenciones en cuanto a la nomenclatura, y volúmenes de datos, se ha elegido un corpus que engloba 10 sistemas de transporte en varios continentes, detallado en el cuadro 4.1. Como medida de la escala, se utiliza el número de líneas de `stop_times.txt`, dado que es el mayor por varios órdenes de magnitud, y por tanto representa la complejidad computacional.

Llevar a cabo las pruebas utilizando estos conjuntos de datos permite asegurar que el sistema funciona correctamente bajo condiciones de internacionalización (codificación UTF-8 para caracteres cirílicos e ideogramas chinos) e independientemente de la topología de red (soportando desde redes de metro densas hasta líneas de autobús regionales).

Nombre	Tipo	Stop Times
Madrid (EMT)	Urbano (bus)	2,23 M
Praga	Regional mixto (bus, metro, tren)	1,88 M
Hong Kong	Urbano mixto (bus, ferry, funicular) Codificación UTF-8 (ideogramas) Alta densidad	1,25 M
Bogotá (SIMUR)	Urbano (bus) Gran número de trayectos (> 150.000)	7,82 M
Singapur	Urbano mixto (bus, metro) Modelo basado en frecuencias Alta densidad	69 k
Galicia (Xunta)	Regional (bus) Contexto conocido Gran número de rutas (> 6.500) Gran número de paradas (> 25.000)	1,65 M
Munich	Urbano mixto (bus, tranvía)	1,39 M
Moscú	Regional (bus) Codificación UTF-8 (cirílico)	2,67 M
Nueva York (MTA)	Urbano (metro) Referencia en el sector	562 k
Belgrado	Urbano mixto (bus, tranvía)	1,88 M

Cuadro 4.1: Conjuntos de datos utilizados para las pruebas.

4.1.4. Metodología

Dado el volumen de los conjuntos de datos elegidos, la estrategia de pruebas busca minimizar la necesidad de definir resultados esperados de manera manual, ya que resulta inabordable. En su lugar, se opta por los siguientes enfoques, complementarios entre sí:

- **Validación cruzada:** Aprovechando la redundancia entre ambos sistemas gestores de bases de datos, PostgreSQL y Neo4J, se verifica que los resultados generados por estos coinciden entre sí, tanto para comprobar la *integridad* de los datos como la *consistencia* en la implementación de las consultas.

- **Análisis de casos límite:** Se evalúa la *robustez* del sistema ante valores de entrada atípicos o extremos, garantizando que no se producen errores al forzar la lógica de implementación de las consultas.
- **Comprobación basada en propiedades:** Con el objetivo de asegurar la *plausibilidad* de los resultados de las consultas, se definen propiedades matemáticas que estos han de cumplir, así como criterios de coherencia basados en el dominio del problema.

4.2. Integridad de los datos

A la hora de comprobar que el proceso de importación se ha realizado con éxito, el enfoque clave es la validación cruzada entre ambos motores de bases de datos. Para llevarla a cabo:

1. Se recupera la información que acaba de ser importada mediante consultas, con sus respectivos JOIN en el caso de PostgreSQL y *pattern matching* en el de Neo4J.
2. Se ordenan los resultados ya en Python, para evitar problemas con las diferencias en el orden proporcionado por las BBDD a causa de la *collation*.
3. Se realiza una comparación extensiva, que asegure tanto que el número de resultados es el mismo, como que son cualitativamente idénticos.

Se puede ver un modelo de implementación de estas pruebas en la figura 4.1, y una muestra de ejecución, sobre el conjunto de datos de Galicia, en la figura 4.2.

4.3. Algoritmos de consulta

A diferencia de las pruebas de importación, para asegurar el correcto funcionamiento de las consultas, debemos elegir un amplio espectro de casos de prueba, que evalúen su desempeño tanto en **condiciones habituales** como en **situaciones atípicas**. Al mismo tiempo, es conveniente comprobar que se mantienen ciertos **invariantes** en los resultados.

Los detalles de implementación se tratan en las siguientes subsecciones, y en la figura 4.5 se proporciona un ejemplo de ejecución.

```

def test_area_data_consistency(pg_query_runner, neo4j_query_runner):
    pg_data = pg_query_runner(
        "SELECT * FROM area ORDER BY area_id;", ())
    neo4j_data = neo4j_query_runner(
        "MATCH (a:Area) RETURN a ORDER BY a.id;", {})

    pg_count, neo4j_count = len(pg_data), len(neo4j_data)

    assert pg_count == neo4j_count

    def to_canonical_tuple(row):
        return (row.get('area_id') or row.get('id'),
                row.get('area_name') or row.get('name'))

    pg_tuples = [to_canonical_tuple(row) for row in pg_data]
    neo4j_tuples = [to_canonical_tuple(row) for row in neo4j_data]

    pg_tuples.sort()
    neo4j_tuples.sort()

    assert pg_tuples == neo4j_tuples

```

Figura 4.1: Muestra de código de las pruebas de importación (simplificada).

```

(.venv) dani@arch ~/C/2/TF6 (master)> pytest Tests/Import/*.py
===== test session starts =====
platform linux -- Python 3.13.11, pytest-8.4.2, pluggy-1.6.0
rootdir: /home/dani/College/2024-2025/TF6
plugins: hypothesis-6.140.0
collected 33 items

Tests/Import/agency.py . [ 3%]
Tests/Import/areas.py . [ 6%]
Tests/Import/attributions.py . [ 9%]
Tests/Import/booking_rules.py .. [ 15%]
Tests/Import/calendar_dates.py . [ 18%]
Tests/Import/calendar~.py . [ 21%]
Tests/Import/fare_attributes.py . [ 24%]
Tests/Import/fare_leg_join_rules.py . [ 27%]
Tests/Import/fare_leg_rules.py . [ 30%]
Tests/Import/fare_media.py . [ 33%]
Tests/Import/fare_products.py . [ 36%]
Tests/Import/fare_rules.py . [ 39%]
Tests/Import/fare_transfer_rules.py . [ 42%]
Tests/Import/feed_info.py . [ 45%]
Tests/Import/frequencies.py . [ 48%]
Tests/Import/levels.py .. [ 54%]
Tests/Import/location_group_stops.py . [ 57%]
Tests/Import/location_groups.py .. [ 63%]
Tests/Import/networks.py . [ 66%]
Tests/Import/pathways.py . [ 69%]
Tests/Import/routes.py . [ 72%]
Tests/Import/shapes.py .. [ 78%]
Tests/Import/stop_areas.py . [ 81%]
Tests/Import/stop_times.py . [ 84%]
Tests/Import/stops.py . [ 87%]
Tests/Import/timeframes.py . [ 90%]
Tests/Import/transfers.py . [ 93%]
Tests/Import/translations.py . [ 96%]
Tests/Import/trips.py . [100%]

===== 33 passed in 281.68s (0:04:41) =====

```

Figura 4.2: Ejecución de las pruebas de importación (*dataset* de Galicia).

4.3.1. Casos de prueba aleatorizados

Para simular consultas que puedan ser realizadas por un usuario final, se implementan casos de prueba generados de manera aleatoria. Sin embargo, estos no son puramente estocásticos, a fin de evitar lanzar numerosas consultas triviales, cuyos resultados siempre sean nulos:

1. **Descubrimiento:** Se consulta la base de datos para determinar el rango de valores válidos, tales como el rango de fechas de servicio, o la *bounding box* del conjunto de paradas.
2. **Selección:** En base a los rangos obtenidos en el paso anterior, se eligen argumentos válidos para crear un caso de prueba.
3. **Consulta y comprobación:** Se lanza el caso de prueba y se verifica que los resultados proporcionados por ambas bases de datos son idénticos.

En cuanto al número de casos de prueba, para cada conjunto de datos, se han considerado 30 para la mayoría de consultas, y 10 para aquellas que requieren un mayor tiempo de ejecución.

4.3.2. Valores límite

Aunque, en la mayoría de situaciones, un usuario final no vaya a realizar consultas con valores que no tengan sentido en el dominio del problema, es importante tener en cuenta dichos casos límite para garantizar la robustez de los algoritmos usados. Algunos ejemplos de argumentos considerados son:

- Fechas inmediatamente anteriores al comienzo del servicio o posteriores al fin de este.
- Coordenadas de lugares aislados como los polos norte y sur.
- Distancias de búsqueda nulas.
- Paradas inexistentes.
- Rutas inexistentes.
- Horas del día tardías.
- Intervalos temporales muy cortos.

Adicionalmente, la figura 4.3 presenta un ejemplo de implementación de estas pruebas, incluyendo varias de las clases de argumentos listados.

```

# Edge cases for stops_within_distance.
def test_edge_cases(pg_query_runner, neo4j_query_runner, bounding_box):
    # North Pole.
    results = run_test_case(origin_lat=90.0, origin_lon=0.0,
                            seek_dist=MAX_SEARCH_DISTANCE_METERS)
    assert len(results) == 0

    # South Pole.
    results = run_test_case(origin_lat=-90.0, origin_lon=0.0,
                            seek_dist=MAX_SEARCH_DISTANCE_METERS)
    assert len(results) == 0

    # Null search distance (multiple iterations).
    for i in range(RANDOM_TEST_COUNT):
        lat, lon = random_point_in_bbox(bounding_box)
        results = run_test_case(origin_lat=lat, origin_lon=lon, seek_dist=0)
        assert len(results) == 0

    # Well outside the bounding box, with long range.
    outside_lat = bounding_box['max_lat'] + 10.0
    outside_lon = bounding_box['max_lon'] + 10.0
    distance = 5000
    results = run_test_case(origin_lat=outside_lat, origin_lon=outside_lon,
                            seek_dist=distance)
    assert len(results) == 0

```

Figura 4.3: Muestra de código de las pruebas de valores límite (simplificada).

4.3.3. Invariantes

Por último, utilizando la biblioteca *hypothesis*[14], se implementan pruebas que tienen como objetivo asegurar el cumplimiento de ciertas propiedades en los resultados de las consultas.

A modo de ejemplo, los resultados de una consulta sobre un rango temporal han de estar contenidos en los resultados de otra consulta sobre otro rango temporal mayor, es decir:

$$\forall a, a', b, b' \in [d_{begin}, d_{end}], a \leq a' \leq b' \leq b \implies q(a', b') \subseteq q(a, b)$$

La implementación de la prueba correspondiente puede verse en la figura 4.4.

Asimismo, durante la ejecución de los casos de prueba aleatorizados, se introducen *comprobaciones de plausibilidad*, como que el número de elementos en listas no es negativo, que los resultados se encuentran dentro del rango de búsqueda, o que estos están ordenados de la manera esperada.

```

# Property: Subrange queries should return the same results as
#           the full range query filtered to that subrange.
@given(
    sub_start=st.dates(
        min_value=service_date_range['min_date'],
        max_value=service_date_range['max_date']
    ),
    sub_end=st.dates(
        min_value=service_date_range['min_date'],
        max_value=service_date_range['max_date']
    )
)
@settings(deadline=None)
def test_pbt_subrange_consistency(full_pg, full_neo, sub_start, sub_end):
    assume(sub_start <= sub_end)

    # Run subrange queries
    sub_pg = pg_query_runner(SQL, (sub_start, sub_end))
    sub_neo = neo4j_query_runner(CYPHER,
        {'start_date': str(sub_start), 'end_date': str(sub_end)})

    # Filter the full-range results to the subrange
    filtered_pg = [r for r in full_pg
        if sub_start <= r['service_date'] <= sub_end]
    filtered_neo = [r for r in full_neo
        if sub_start <= r['value']['service_date'] <= sub_end]

    # Check if the property holds.
    assert sub_pg == filtered_pg
    assert sub_neo == filtered_neo

```

Figura 4.4: Muestra de código de las pruebas de invariantes (simplificada).

```

(.venv) dani@garch ~/C/2/TF6 (master)> pytest Tests/*.py
===== test session starts =====
platform linux -- Python 3.13.11, pytest-8.4.2, pluggy-1.6.0
rootdir: /home/dani/College/2024-2025/TF6
plugins: hypothesis-6.140.0
collected 29 items

Tests/active_services.py ... [ 10%]
Tests/daily_status.py ... [ 20%]
Tests/departure_times.py ... [ 31%]
Tests/headway_stats.py ... [ 41%]
Tests/next_departures.py ... [ 51%]
Tests/overlapping_segments.py . [ 55%]
Tests/routes_by_relevance.py ... [ 65%]
Tests/routes_by_speed.py . [ 68%]
Tests/stops_within_distance.py ... [ 79%]
Tests/top_stops.py ... [ 89%]
Tests/trip_start_time_distribution.py ... [100%]

===== 29 passed in 380.94s (0:06:20) =====

```

Figura 4.5: Ejecución de las pruebas de consultas (*dataset* de Galicia).

Capítulo 5

Evaluación comparativa

Una vez llevado a cabo el proceso de pruebas, y garantizado el correcto funcionamiento del sistema, este capítulo aborda las diferencias entre las implementaciones objeto-relacional (PostgreSQL) y orientada a grafos (Neo4J).

El análisis se centra en tres dimensiones: eficiencia (tiempos de ejecución del proceso de importación y de las consultas), facilidad de uso (modelado y expresión de consultas) y soporte de bibliotecas.

5.1. Eficiencia

Durante el proceso de pruebas, se llevan a cabo mediciones de los tiempos de ejecución, utilizadas en esta comparativa, para todos los conjuntos de datos descritos en la figura 4.1. En el caso de las consultas, se han medido sus tiempos de ejecución en los casos de prueba aleatorizados (sección 4.3.1).

5.1.1. Tiempo de importación (input)

El proceso de ingesta de datos muestra las mayores diferencias en términos de rendimiento, como se puede comprobar en la figura 5.1. En general, PostgreSQL es **un orden de magnitud** más rápido que Neo4J, siendo el *speedup* alcanzado de casi 13x.

A este respecto, conviene mencionar que los procesos de importación en ambos sistemas gestores de bases de datos son fundamentalmente diferentes: mientras que PostgreSQL copia los datos directamente de los ficheros .csv, y a continuación comprueba la integridad referencial y que los datos sean válidos en el dominio, Neo4J también ha de construir la topología de la red, lo que implica determinar los nodos origen y destino y almacenar la relación entre ellos.

Por otra parte, cabe mencionar que Neo4J no proporciona soporte¹ para copiar directamente datos desde un archivo .csv lo que obliga a leer las líneas de manera secuencial, agruparlas en conjuntos de miles de líneas (para evitar desbordamientos de memoria), y crear los nodos correspondientes.

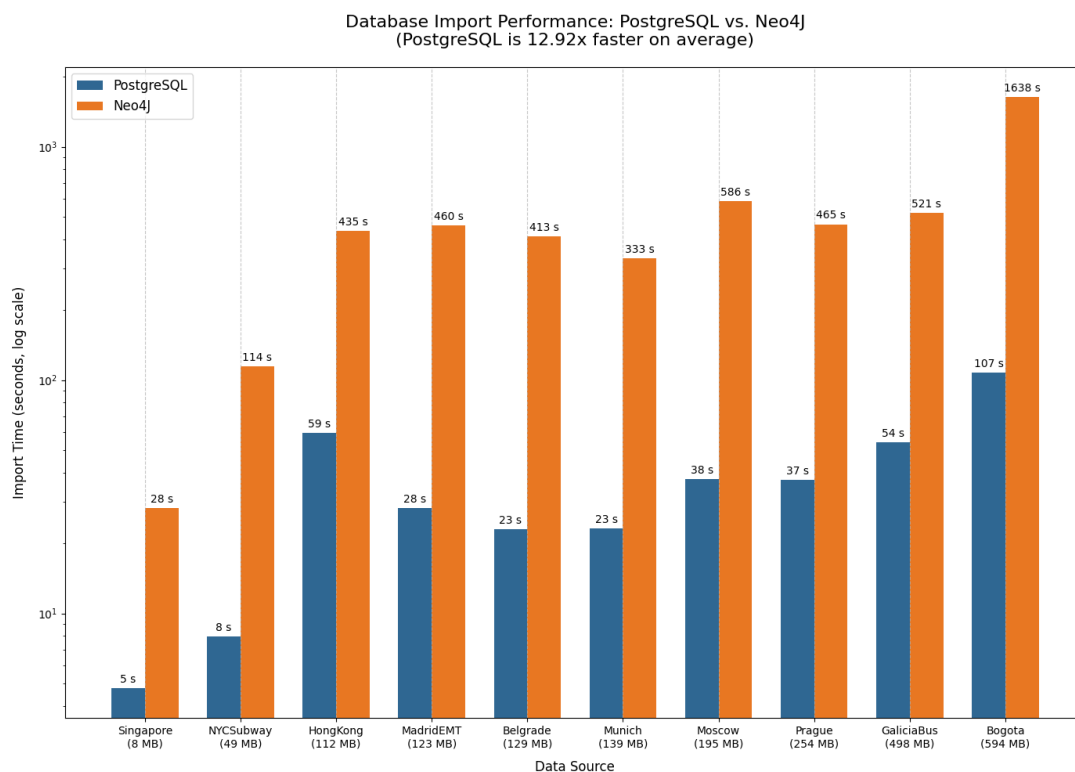


Figura 5.1: Tiempos de importación en PostgreSQL y Neo4J.

5.1.2. Tiempo de consulta (output)

En el ámbito de la recuperación y análisis de datos, las diferencias de rendimiento son menores, y, además, dependen de la consulta realizada, como se puede apreciar en la figura 5.2. A causa de ello, no tiene sentido considerar el speedup global, dado que no hay un sistema que rinda mejor de manera clara. Comentar, además, que la gráfica no muestra ciertas consultas que no se han podido implementar en Neo4J únicamente utilizando Cypher (véase 6.2), como las correspondientes a los requisitos RF-15 y RF-16.

¹Existe una herramienta externa, neo4j-admin, que permite cargar ficheros .csv directamente, pero estos deben ser adaptados a un formato específico. Es posible que el tiempo de importación se pueda reducir de esta manera, pero ello complica el procesamiento de datos.

Resulta interesante destacar que Neo4J alcanza un mejor rendimiento en aquellas consultas que requieren atravesar múltiples niveles de relaciones desde un número reducido de nodos iniciales, siendo un ejemplo `next_departures`, que determina las próximas salidas desde una parada, lo que implica recorrer la topología para examinar todos los trayectos que parten de ella. Esto se debe a que los nodos relacionados se almacenan cerca en memoria (*almacenamiento basado en topología*), optimizando dichos recorridos.

A su vez, PostgreSQL proporciona menores tiempos de ejecución en consultas que involucran cantidades masivas de datos, como `overlapping_segments`, que necesita considerar cada uno de los tiempos de llegada de cada trayecto, gracias a que realiza un único JOIN secuencial entre las tablas (es decir, conjuntos de tuplas del mismo tipo), y no millones de accesos aleatorios a memoria.

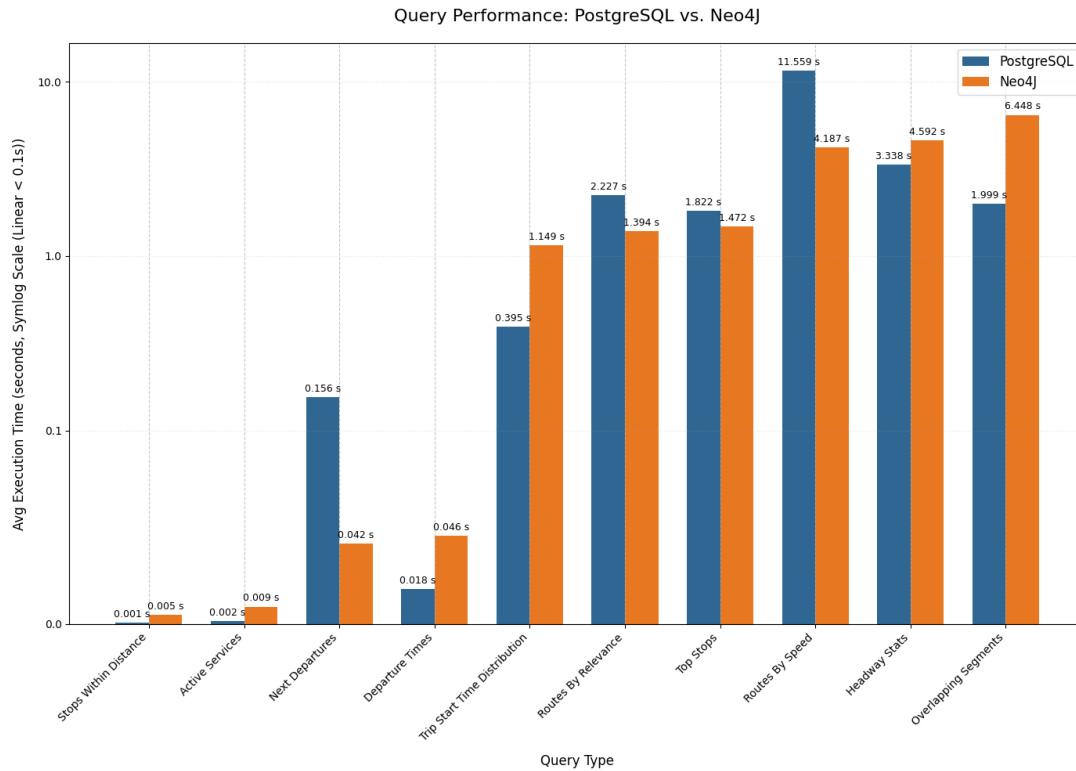


Figura 5.2: Tiempos de consulta en PostgreSQL y Neo4J.

5.2. Modelado y expresión

En ocasiones, resulta preferible potenciar la productividad de los desarrolladores incluso en detrimento del rendimiento que el sistema final pueda ofrecer.

Por ello, otro de los aspectos a considerar en esta comparativa es cómo de sencillo resulta tanto modelar la información del sistema como expresar las consultas.

En cuanto al primer aspecto, y dado que la amplia mayoría de la información que debe almacenar la base de datos viene dada por el estándar GTFS, PostgreSQL permite un modelado más rápido y sencillo, dado que los campos en la especificación están definidos siguiendo el paradigma de las bases de datos relacionales.

Como muestra la figura 5.3, PostgreSQL permite almacenar de manera directa la información tal y como se define en dicha especificación, mientras que Neo4J requiere adaptar el modelo y decidir, por ejemplo, si transformar cada clave foránea de la especificación en una relación entre nodos, e incluso en el caso de proceder de ese modo por simplicidad, la ausencia de valores por defecto obliga a introducir lógica condicional.

```
CREATE TABLE trip (
  route_id TEXT NOT NULL
    REFERENCES route(route_id),
  service_id TEXT NOT NULL,
  trip_id TEXT PRIMARY KEY,
  trip_headsign TEXT,
  trip_short_name TEXT,
  direction_id INTEGER
    REFERENCES travel_direction(id),
  block_id TEXT,
  shape_id TEXT,
  wheelchair_accessible INTEGER
    DEFAULT 0
    REFERENCES wheelchair_status(id),
  bikes_allowed INTEGER
    DEFAULT 0
    REFERENCES bicycle_status(id)
);

MATCH (r: Route { id: row.route_id })
MATCH (s: Service { id: row.service_id })
WITH row, r, s
CREATE (t: Trip {
  id: row.trip_id,
  headsign: row.trip_headsign,
  short_name: row.trip_short_name
})
CREATE (t)-[fol: FOLLOWS]->(r)
CREATE (t)-[sch: SCHEDULED_BY]->(s)
CREATE (t)-[hws: HAS_WHEELCHAIR_STATUS]->(ws)
CREATE (t)-[hbs: HAS_BICYCLE_STATUS]->(bs)
WITH row, t
CALL apoc.do.when(
  row.direction_id IS NOT NULL,
  'CREATE (t)-[htd: HAS_TRAVEL_DIR]->(td)'
  RETURN row, t',
  'RETURN row, t',
  { row: row, t: t }
) YIELD value
WITH value.row as row, value.t as t
CALL apoc.do.when(
  row.block_id IS NOT NULL,
  'CREATE (t)-[itb: IN_TRIP_BLOCK]->(tb)'
  RETURN row, t',
  'RETURN row, t',
  { row: row, t: t }
) YIELD value
```

Figura 5.3: Comparativa de modelado entre PostgreSQL (izq.) y Neo4J (der.).

Sin embargo, Neo4J ofrece una sintaxis que se adecúa mejor a la expresión de múltiples consultas, como es el caso de `overlapping_segments` (figura 5.5). Si comparamos esta implementación con la de PostgreSQL (figura 5.4), vemos que Neo4J requiere menos código y este resulta más fácil de entender, dado que su lenguaje está específicamente diseñado para aplicar múltiples transformaciones consecutivas a los datos (*pipelining*).

```

WITH trip_segments AS (
  SELECT
    trip_id,
    stop_id AS from_stop_id,
    LEAD(stop_id, 1) OVER
      (PARTITION BY trip_id ORDER BY stop_sequence) AS to_stop_id
  FROM stop_time
),
segment_routes AS (
  SELECT DISTINCT
    ts.from_stop_id,
    ts.to_stop_id,
    t.route_id
  FROM trip_segments AS ts
  JOIN trip AS t ON ts.trip_id = t.trip_id
  WHERE ts.to_stop_id IS NOT NULL
)
SELECT
  s1.stop_name AS from_stop,
  s2.stop_name AS to_stop,
  COUNT(sr.route_id) AS route_count,
  array_agg(r.route_short_name ORDER BY r.route_short_name) AS routes
FROM segment_routes AS sr
  JOIN stop AS s1 ON sr.from_stop_id = s1.stop_id
  JOIN stop AS s2 ON sr.to_stop_id = s2.stop_id
  JOIN route AS r ON sr.route_id = r.route_id
GROUP BY sr.from_stop_id, sr.to_stop_id, s1.stop_name, s2.stop_name
ORDER BY route_count DESC, from_stop, to_stop;

```

Figura 5.4: Código de `overlapping_segments` en PostgreSQL.

```

MATCH (s:Stop)<-[:LOCATED_AT]-(st:StopTime)-[:PART_OF]->(t:Trip)
WITH t, s, st
ORDER BY st.stop_sequence

WITH t, apoc.coll.pairsMin(collect(s)) as stop_sequences
MATCH (t:Trip)-[:FOLLOWS]->(r:Route)
UNWIND stop_sequences as stop_pair
WITH DISTINCT r, stop_pair[0] as from_stop, stop_pair[1] as to_stop

WITH from_stop, to_stop, apoc.coll.sort(collect(r.short_name)) AS routes
ORDER BY size(routes) DESC, from_stop.name, to_stop.name
RETURN from_stop.name AS from_stop,
  to_stop.name AS to_stop,
  size(routes) AS route_count,
  routes

```

Figura 5.5: Código de `overlapping_segments` en Neo4J.

5.3. Soporte de bibliotecas

Otro de los aspectos clave a considerar a nivel de productividad del programador es el soporte de bibliotecas, dado que estas simplifican en gran medida la implementación de consultas complejas.

En el caso de PostgreSQL, su extensión PostGIS es un estándar en la industria y ofrece una gran cantidad de funcionalidades, desde las más básicas, como la creación de puntos y líneas, hasta otras más avanzadas como el cálculo de elementos geométricos, como la envoltura cóncava y convexa (*concave/convex hull*) y la generación de diagramas de Voronoi, que permiten derivar áreas de servicio a partir de un conjunto de paradas, o la segmentación de mapas en regiones hexagonales para generar mapas de calor.

En cambio, Neo4J Spatial es una biblioteca más simple y con funcionalidades muy restringidas, ofreciendo, en el contexto de una red de transportes, poco más que creación de puntos y líneas, y cálculo de distancias. Precisamente a causa de ello, se considera inviable la implementación de ciertas consultas, como `stop_density_heatmap`, en Cypher, y no se lleva a cabo.

5.4. Resultados

A modo de conclusión del capítulo, el cuadro 5.1 resume los puntos fuertes y débiles identificados en cada uno de los motores de base de datos.

Dimensión	Vencedor	Observaciones
Importación	PostgreSQL	Alrededor de un orden de magnitud más rápido.
Consulta	—	PostgreSQL rinde mejor en agregación de cantidades masivas de datos, Neo4J en travesía desde un número limitado de nodos iniciales.
Modelado	PostgreSQL	GTFS se define siguiendo el modelo relacional.
Expresividad	Neo4J	Cypher resulta más simple y fácil de entender.
Bibliotecas	PostgreSQL	PostGIS soporta más algoritmos geométricos.

Cuadro 5.1: Resumen comparativo: PostgreSQL frente a Neo4J.

Capítulo 6

Conclusiones y posibles ampliaciones

Este capítulo final sintetiza los resultados obtenidos durante el desarrollo del proyecto, y propone líneas de investigación para futuras mejoras.

6.1. Hallazgos

Este trabajo ha permitido contrastar dos paradigmas de bases de datos diferentes, aplicados a un mismo dominio: la gestión y análisis de redes de transporte público.

La hipótesis inicial sugería que, dada la naturaleza topológica de una red de transporte, que cuenta con nodos y conexiones, una base de datos orientada a grafos como Neo4J debería ofrecer un rendimiento superior. Sin embargo, los resultados empíricos obtenidos matizan esta premisa, dado que:

1. PostgreSQL ha ofrecido un mejor rendimiento en tareas de ingesta de datos, particularmente a nivel de importación, siendo un orden de magnitud más rápido que Neo4J.
2. El tiempo de consulta varía en función de la naturaleza de esta, siendo Neo4J más rápido a la hora de ejecutar consultas que requieren atravesar múltiples niveles de relaciones desde un número reducido de nodos iniciales, mientras que PostgreSQL vence en consultas que requieren agregar cantidades masivas de datos.
3. PostgreSQL (y todas las bases de datos relacionales, en general) simplifica el proceso de modelado, al estar el estándar GTFS definido siguiendo el modelo relacional.

4. Neo4J permite expresar las consultas de manera más sencilla y breve, lo que facilita su mantenimiento y comprensión.
5. El soporte de bibliotecas, fundamental en este dominio, es excelente en PostgreSQL, y bastante más limitado en Neo4J, lo que ha impedido la implementación de ciertas consultas (véase 6.2).

En definitiva, Neo4J es un sistema sencillo de utilizar y flexible, pero que fuera de consultas con requisitos muy específicos, ofrece un rendimiento peor que PostgreSQL. Al mismo tiempo, y aunque una de las ventajas de Neo4J es la mejora en productividad que ofrece al desarrollador a la hora de escribir consultas, esta se ve opacada por el reducido soporte geoespacial de Neo4J Spatial, especialmente en comparación con PostGIS.

6.2. Grado de cumplimiento de los objetivos

A grandes rasgos, se considera que los objetivos principales del proyecto se han alcanzado de manera satisfactoria:

- Desarrollo de un sistema capaz de importar y validar datos que siguen el estándar GTFS, abarcando desde redes de transporte locales hasta regionales.
- Implementación de un catálogo de consultas predefinidas que el usuario puede lanzar proporcionando únicamente sus argumentos.
- Visualización de los resultados tanto mediante integraciones con Python, utilizando Folium, como con herramientas GIS de escritorio como QGIS.
- Realización de una comparativa de las ventajas que ofrece cada uno de los motores de base de datos en el contexto de este dominio.

Sin embargo, cabe mencionar que debido a ciertas limitaciones en el soporte de bibliotecas de Neo4J, no se han podido implementar algunas consultas, como `stop_density_heatmap`, que requiere la segmentación del mapa en regiones. Asimismo, dado que resulta imposible expresar un algoritmo imperativo en Cypher, no se ha podido implementar `earliest_arrivals`, y sería necesario crear un plugin externo, en Java, para ello.

6.3. Posibles vías de mejora

6.3.1. Soporte para GTFS Realtime

Una de las evoluciones naturales del sistema consistiría en añadir soporte para datos en tiempo real, siguiendo el estándar GTFS Realtime[4] (GTFS-RT). Actualmente, el sistema considera una planificación estática, siguiendo horarios teóricos, lo que limita su utilidad en ciertos contextos, como, por ejemplo, frente a retrasos a causa de atascos o incidentes.

Además, la integración de flujos de datos en vivo plantearía nuevos desafíos arquitectónicos, principalmente derivados de la alta frecuencia de escritura, o la actualización de los resultados proporcionados por algoritmos de enrutamiento.

6.3.2. Análisis de topología y resiliencia de la red

Aunque en este trabajo se han abordado ciertos aspectos del análisis basado en topología, como la identificación de paradas consecutivas servidas por un número elevado de rutas (`overlapping_segments`), este análisis podría extenderse a otras áreas, como por ejemplo:

- Detección de nodos críticos mediante algoritmos de centralidad, como *Betweenness Centrality*[18], para encontrar cuellos de botella.
- Simulación de fallos en la red mediante la eliminación de nodos o aristas para evaluar cómo se degrada la conectividad y determinar la robustez ante incidentes.
- Identificación de comunidades mediante algoritmos de *clustering* para hallar zonas fuertemente conectadas entre sí pero aisladas del resto.

6.3.3. Enrutamiento con restricciones multicriterio

Otra ampliación relevante sería la implementación de algoritmos de enrutamiento que consideren restricciones complejas por parte del usuario, más allá de minimizar el tiempo de viaje. El ejemplo más claro es la introducción de límites en la distancia recorrida a pie (llegar al destino caminando menos de X metros).

Desde el punto de vista algorítmico, este cambio transformaría el problema en un *Resource Constrained Shortest Path Problem* (RCSP), que, de manera general, pertenece a la clase de complejidad NP-hard, por lo que sería necesario hacer uso de aproximaciones heurísticas que, potencialmente, producirían resultados subóptimos pero *suficientemente buenos*.

Apéndice A

Manual técnico

A.1. Código fuente

Todo el código fuente del proyecto se encuentra disponible en el repositorio de GitHub asociado.

El proyecto se compone mayormente de tres partes: scripts en Python (para adaptar los datasets, lanzar el proceso de importación, generar gráficas y mapas con los resultados de las consultas, y ejecutar pruebas), código SQL (para generar las tablas en PostgreSQL, cargar datos y crear las consultas predefinidas), y código Cypher (con el mismo objetivo que el SQL, pero en Neo4J).

A.2. Requisitos, dependencias e instalación

Los requisitos principales del sistema y las dependencias se encuentran descritos en las secciones B.1 y B.2 del manual de usuario. A mayores de lo necesario para instalar y ejecutar el software, se recomienda contar con un editor de texto o entorno de desarrollo con soporte para Python, SQL y Cypher, como Visual Studio Code.

La instalación, incluyendo el proceso de clonar el repositorio de GitHub, descargar las bibliotecas de Python necesarias e importar un dataset, se trata en el apartado B.3 del manual de usuario. Dado que el código está formado únicamente por scripts y no necesita compilación, el proceso de instalación para un desarrollador y un usuario es el mismo.

A.3. Estructura del proyecto

Los directorios del proyecto se disponen de la siguiente manera:

```

TFG
├── Datasets
│   └── GTFS ..... Conjuntos de datos, cada uno en un subdirectorio
├── Examples ..... Imágenes y mapas interactivos (HTML) de muestra
├── Scripts ..... Scripts para importación y visualización
│   ├── Neo4J
│   │   ├── Import ..... Carga de datos
│   │   │   ├── calendar.cypher
│   │   │   ├── routes.cypher
│   │   │   ├── stops.cypher
│   │   │   └── ...
│   │   ├── launch.sh ..... Lanzamiento de Neo4J sobre Docker
│   │   └── queries.cypher ..... Definición de consultas
│   ├── PostgreSQL
│   │   ├── Import ..... Carga de datos
│   │   │   ├── calendar.sql
│   │   │   ├── routes.sql
│   │   │   ├── stops.sql
│   │   │   └── ...
│   │   ├── launch.sh ..... Lanzamiento de PostgreSQL sobre Docker
│   │   └── queries.sql ..... Definición de consultas
│   ├── database.py ..... Conexiones a las BBDD desde Python
│   ├── departure_sign_animation.py
│   ├── headway_stats_graph.py
│   ├── import.py ..... Script de importación para el usuario
│   ├── process_dataset.py ..... Adaptación de datasets
│   ├── route_speed_interactive_map.py
│   ├── shortest_path_interactive_map.py
│   └── ...
├── Tests ..... Scripts de pruebas
│   ├── Import ..... Pruebas de importación
│   ├── active_services.py
│   ├── conftest.py ..... Utilidades para las pruebas
│   ├── daily_status.py
│   ├── departure_times.py
│   └── ...
├── Volumes ..... Datos de Postgres y Neo4J, incluyendo plugins
├── LICENSE ..... Licencia del código (GPLv3)
└── requirements.txt ..... Lista de dependencias para pip

```

A.4. Modificaciones

A.4.1. Actualizaciones en el esquema

El esquema de ambas bases de datos se define en el código de importación, presente en `Scripts/PostgreSQL/Import` y `Scripts/Neo4J/Import` respectivamente. Cada uno de los archivos del estándar GTFS cuenta con su propio script, nombrado de la misma forma, pero con la extensión correspondiente. Por ejemplo, para `agency.txt`, tenemos los scripts `agency.sql` y `agency.cypher`.

Si únicamente queremos introducir cambios relativos a entidades que ya existen en las bases de datos, es suficiente con modificar los scripts mencionados, pero si la intención es añadir soporte para archivos adicionales, debemos:

1. Crear nuestro propio script y nombrarlo siguiendo la convención previa.
2. Escribir el código siguiendo la estructura los demás scripts de importación.
3. Modificar el fichero `Scripts/import.py`, incluyendo el nombre del nuevo script en la variable `file_list`. Cabe mencionar que el orden de ejecución de los scripts es el mismo que el de la lista, por lo que debemos tener en cuenta si hay dependencias respecto a otras entidades.
4. (*Opcional, pero recomendado*) Siguiendo la metodología de pruebas, escribir tests de validación cruzada, que aseguren que los datos importados en ambos sistemas son equivalentes, en `Tests/Import/<nombre_script>.py`.

A.4.2. Adición de consultas

Las consultas predefinidas se encuentran en `queries.sql` y `queries.cypher` bajo los directorios `Scripts/PostgreSQL` y `Scripts/Neo4J` respectivamente. Si queremos implementar nuevas consultas, los pasos para ello son:

1. Escribir el código SQL y Cypher de la consulta en cada uno de los archivos.
2. En el fichero `Scripts/database.py`, añadir el nombre de la consulta y sus parámetros al diccionario `QUERY_PARAMETERS`, siguiendo el formato del resto de consultas. Este paso permite lanzar la query desde un script en Python.
3. (*Opcional, pero recomendado*) Siguiendo la metodología de pruebas, escribir tests de validación cruzada, análisis de casos límite y comprobación de invariantes (propiedades) en el fichero `Tests/<nombre_query>.py`.

A.5. Ejecución de pruebas

Una vez modificado el código, resulta deseable ejecutar pruebas que verifiquen que nuestro cambio no ha tenido efectos inesperados en el resto del sistema. Para ello, podemos utilizar `pytest` (ya instalado como parte de las dependencias del proyecto), y lanzar todos los tests desde el directorio raíz del proyecto:

```
$ pytest -v Tests/*.py Tests/Import/*.py
```

Es importante mencionar que, dado que las pruebas utilizan validación cruzada entre ambos sistemas gestores de bases de datos, debemos asegurarnos de **haber lanzado ambos, e importado el mismo dataset**.

Para más información acerca de la metodología de pruebas seguida, se puede consultar el capítulo 4.

A.6. Errores frecuentes

A.6.1. Las pruebas producen errores de conexión

Al tratar de ejecutar un script mediante `pytest`, podemos encontrar un mensaje de error similar al siguiente:

```
psycopg.OperationalError: connection failed: connection to server  
at "127.0.0.1", port 5432 failed: Connection refused
```

En ese caso, debemos asegurarnos de que los contenedores Docker de PostgreSQL y Neo4J se están ejecutando:

```
$ Scripts/PostgreSQL/launch.sh  
$ Scripts/Neo4J/launch.sh
```

También conviene asegurarse de que ningún servicio está tratando de utilizar los mismos puertos, 5432 en el caso de PostgreSQL, y 7474 y 7687 en el de Neo4J.

A.6.2. Otros errores

Dado que durante el uso regular del sistema se llevan a cabo procesos muy similares a los que encontraríamos en el caso del desarrollo, es recomendable consultar el apartado B.6 en el manual de usuario, en que se abordan varios otros errores que podrían aparecer.

Apéndice B

Manual de usuario

B.1. Especificaciones

Se recomienda al usuario contar con un equipo de las siguientes especificaciones, o mejores:

- **CPU:** Intel i7 de octava generación, o su equivalente en AMD.
- **RAM:** 16 GB, dado que el proceso de importación de datos carga mucha información en memoria.
- **Disco duro:** 10 GB libres, en un SSD.
- **Sistema operativo:** Cualquier distribución de Linux, dado que el código proporcionado está diseñado para funcionar con Docker.

Cabe destacar que, aunque no se garantiza, es posible que se pueda emplear el sistema en condiciones diferentes, como, por ejemplo, en Windows, o contando con una menor cantidad de RAM.

B.2. Dependencias

A nivel de software, es necesario contar con los programas indicados en el cuadro B.1 instalados, y, como recomendación, en las versiones señaladas.

Aunque se hace uso de varios sistemas gestores de bases de datos, no es necesario instalarlos de forma explícita, dado que los scripts proporcionados se encargan de lanzar contenedores Docker que los ejecutan directamente, descargando todo lo requerido.

Nombre	Versión
Bash	5.2
Coreutils	9.7
Docker	28.1.1
Git	2.49
Python	3.13
QGIS	3.42

Cuadro B.1: Lista de dependencias software.

B.3. Instalación

Para llevar a cabo la instalación del sistema:

1. Abrir el terminal y navegar hasta el directorio en que queramos instalar el sistema.
2. Clonar el repositorio de GitHub haciendo uso del comando:

```
$ git clone https://github.com/rodrada/TFG
```

3. Asegurarse de que las bibliotecas de Python están instaladas ejecutando, en el directorio raíz del proyecto:

```
$ python3 -m venv .venv
$ source .venv/bin/activate
$ pip install -r requirements.txt
```

Como resultado, el terminal debería mostrar el prefijo (.venv).

4. Descargar el dataset que queramos emplear, de webs como Mobility Database. Para esta guía, utilizaremos el de Singapur.
5. Descomprimir el dataset y colocarlo en su propio subdirectorio bajo el directorio Datasets/GTFS del proyecto.
6. Asegurar que el dataset sigue el formato esperado mediante el script process_dataset.py en el directorio Scripts del proyecto. Por ejemplo:

```
$ ./process_dataset.py ../Datasets/GTFS/Singapore
```

7. Utilizando los scripts `launch.sh` en `Scripts/Neo4J` o `Scripts/PostgreSQL`, lanzar el sistema gestor de bases de datos que queramos utilizar.
8. Por último, ejecutar el script `import.sh` en el directorio `Scripts` del proyecto, indicando qué dataset queremos importar y en qué sistema gestor de bases de datos. Por ejemplo:

```
$ ./import.sh Singapore neo4j
```

B.4. Consultas

B.4.1. Queries predefinidas

Durante el proceso de importación, se añaden múltiples consultas predefinidas que podemos lanzar de manera sencilla, tal y como se explicará en las próximas subsecciones. En el cuadro B.2 podemos ver las consultas disponibles en ambos sistemas, y, en el B.3, las que solo se pueden ejecutar en PostgreSQL.

B.4.2. PostgreSQL

Una vez instalado el sistema, podemos empezar a realizar consultas sobre la información almacenada. En el caso de PostgreSQL, procederemos del siguiente modo:

1. En el terminal, lanzar el cliente en línea de comandos, `psql`, utilizando el siguiente comando:

```
$ docker exec -it postgres psql -U postgres -d gtfs
```

Mediante el mismo, se ejecuta una sesión interactiva de `psql` dentro del contenedor Docker, utilizando el usuario `(-U)` “postgres” y la base de datos `(-d)` “gtfs”.

2. Escribir la consulta que queramos realizar o, en caso de que sea una de las predefinidas, invocarla utilizando la siguiente línea:

```
SELECT * FROM <nombre_consulta>(arg1 => valor1, ...);
```

También se puede omitir el nombre de los argumentos. Por ejemplo:

```
SELECT * FROM stops_within_distance(1.28, 103.82, 500);
```

En la figura B.1, podemos ver un ejemplo de una consulta lanzada en PostgreSQL y el resultado obtenido.

Nombre y argumentos	Descripción
<code>active_services(curr_date: Date)</code>	Enumera los servicios de transporte activos en la fecha <code>curr_date</code> .
<code>daily_status(start_date: Date end_date: Date)</code>	Muestra la cantidad de trayectos, rutas y paradas activos en cada una de las fechas entre <code>start_date</code> y <code>end_date</code> .
<code>departure_times(route_id: String stop_id: String curr_date: Date)</code>	Determina los tiempos de salida para la ruta <code>route_id</code> , desde la parada <code>stop_id</code> , en la fecha <code>curr_date</code> .
<code>headway_stats(curr_date: Date)</code>	Genera estadísticas de los tiempos de espera (percentil 5, mediana, percentil 95 y desviación típica) para todas las rutas en la fecha <code>curr_date</code> .
<code>next_departures(stop_id: String curr_date: Date curr_time: Time)</code>	Encuentra las próximas salidas desde la parada <code>stop_id</code> , partiendo de la fecha <code>curr_date</code> y la hora <code>curr_time</code> .
<code>overlapping_segments()</code>	Busca los segmentos (pares de paradas) de la red que son cubiertos por una mayor cantidad de rutas, lo que puede indicar planificación redundante.
<code>routes_by_relevance(curr_date: Date curr_time: Time)</code>	Dadas la fecha <code>curr_date</code> y la hora <code>curr_time</code> , halla las rutas más importantes en términos de trayectos activos y frecuencia entre estos.
<code>routes_by_speed()</code>	Produce un ranking de las rutas, ordenadas por su velocidad media (considerando el promedio de todos sus trayectos).
<code>stops_within_distance(origin_lat: Float origin_lon: Float seek_dist: Float)</code>	Determina las paradas que se encuentran a menos de <code>seek_dist</code> metros del punto con latitud <code>origin_lat</code> y longitud <code>origin_lon</code> , ordenadas por distancia.
<code>top_stops(curr_date: Date)</code>	Encuentra las paradas más importantes en la fecha <code>curr_date</code> , de acuerdo con el número de salidas.
<code>trip_start_time_distribution(curr_date: Date bucket_size_min: Integer)</code>	Genera un histograma que aglutina las fechas de inicio de los trayectos, en intervalos de <code>bucket_size_min</code> minutos, para la fecha <code>curr_date</code> .

Cuadro B.2: Consultas disponibles en ambos sistemas.

Nombre y argumentos	Descripción
<code>stop_density_heatmap(grid_size_meters: Integer)</code>	Divide el mapa en hexágonos de lado <code>grid_size_meters</code> y determina el número de paradas contenidas en cada uno de ellos.
<code>earliest_arrivals(origin_stop_id: String departure_date: Date departure_time: Time)</code>	Considerando la parada con identificador <code>origin_stop_id</code> , la fecha <code>departure_date</code> y la hora de salida <code>departure_time</code> , calcula el instante de llegada más temprano al resto de paradas de la red.
<code>shortest_path(origin_stop_id: String destination_stop_id: String departure_date: Date departure_time: Time)</code>	Determina el camino más corto (el que permite llegar antes) desde la parada con identificador <code>origin_stop_id</code> hasta la de identificador <code>destination_stop_id</code> , en la fecha <code>departure_date</code> y con la hora de salida <code>departure_time</code> .
<code>route_straightness()</code>	Para cada una de las rutas, determina su índice de rectitud, es decir, el ratio $km_recorridos/km_linea_recta$, lo que puede sugerir ineficiencias (si es reducido) o planificación redundante (si es próximo a 1).

Cuadro B.3: Consultas exclusivas de PostgreSQL.

B.4.3. Neo4J

Neo4J ofrece una alternativa más amigable al usuario que la línea de comandos: una interfaz web, visible en la figura B.2, que, por defecto, se lanza en el puerto 7474.

Para llevar a cabo consultas en Neo4J, se debe:

1. Lanzar el navegador e introducir la IP de la interfaz web:

```
127.0.0.1:7474
```

2. En la ventana de autenticación que aparece al acceder, pulsar en “Connect” **sin introducir credenciales**.
3. Abrir la sección “Database Information”, correspondiente al primer icono en la barra lateral de la izquierda.

```
gtfs=# SELECT * FROM stops_within_distance(1.28, 103.82, 500);
```

id	name	lat	lon	distance	geom
10339	Aft Bt Merah Ctr'l	1.281158	103.818573	204	0101000020E6100000DF18028063F45940EA23F0879F7FF43F
10331	Bef Bt Merah Ctr'l	1.281363	103.818344	238	0101000020E61000002E3883BF5FF4594005DF347D7680F43F
14269	Blk 10	1.277585	103.819995	269	0101000020E61000008CF84ECC7AF4594037C30DF8FC70F43F
14259	Blk 11	1.277845	103.818658	282	0101000020E6100000F62686E464F45940C3B645990D72F43F
14249	Blk 1	1.279474	103.81733	303	0101000020E61000008A027D224FF459401E51A18A8978F43F
14241	Opp Blk 1	1.279144	103.816873	360	0101000020E610000011FFB0A547F4594076FEEDB25F77F43F
10081	Opp Blk 120	1.282278	103.822417	369	0101000020E61000002AABE97AA2F45940A915A6EF3584F43F
10089	Blk 119	1.282923	103.821657	374	0101000020E61000003E13D0796F45940A9F94844D4A8F43F
10009	Bt Merah Int	1.282102	103.817225	387	0101000020E61000004F1E166A4DF45940D82D02637D83F43F
10361	Cendex Ctr	1.280381	103.82353	395	0101000020E610000098512C87B4F4594067B796C9707CF43F
14281	Safra Mt Faber	1.276997	103.818044	398	0101000020E610000019AC38D55AF4594019710168946EF43F
10369	Opp Cendex Ctr	1.280375	103.823873	433	0101000020E6100000ADC3D1558AF4594023DBF97E6A7CF43F
14279	Blk 25	1.275916	103.819209	463	0101000020E6100000B0E595EB6DF45940161747E5266AF43F
14271	Al-Amin Mque	1.275682	103.819149	489	0101000020E61000004563EDEF6CF45940B18861873169F43F
10091	Bt Merah Town Ctr	1.283941	103.81784	500	0101000020E61000001557957D57F4594086E5CFB7058BF43F

(15 rows)

Figura B.1: Ejemplo de consulta en PostgreSQL.

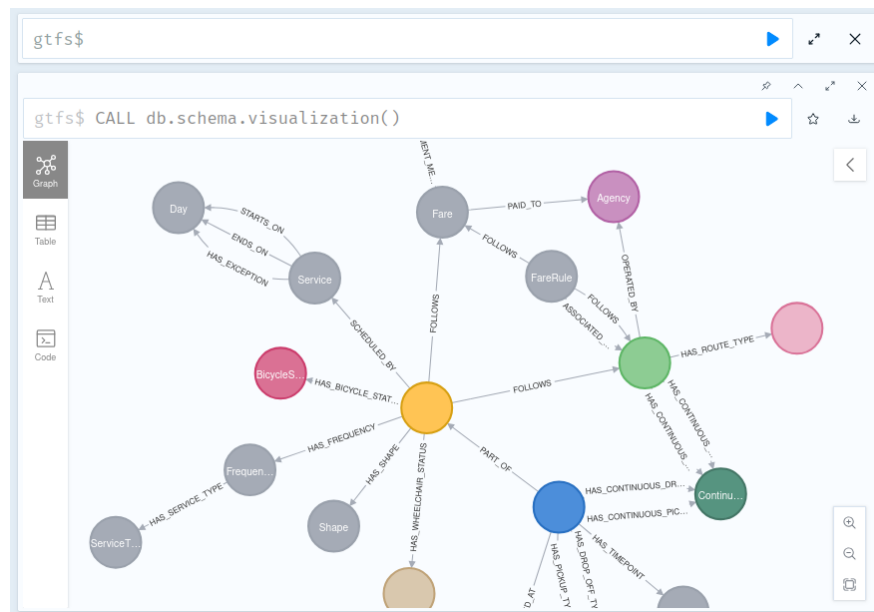


Figura B.2: Interfaz web de Neo4J.

4. Bajo el texto “Use database”, abrir el desplegable y reemplazar el valor por defecto, `neo4j`, por `gtfs`.
5. En el cuadro de texto superior, con el mensaje `gtfs$` (véase la figura B.2), introducir el texto de la consulta que queramos realizar. Para lanzar una consulta predefinida, debemos utilizar la siguiente sintaxis:

```
MATCH (cq: CypherQuery {name: '<nombre_consulta>'})
CALL apoc.cypher.run(cq.statement, {arg1: valor1, ...})
YIELD value
RETURN value;
```

6. Pulsar en el triángulo azul a la derecha del cuadro para lanzar la consulta.

7. Una vez obtenidos los resultados, seleccionar a la izquierda el modo de presentación de estos: grafo, tabla o texto.

B.5. Visualización sobre mapa

B.5.1. QGIS

Si estamos utilizando PostgreSQL como sistema gestor de bases de datos, podemos visualizar los resultados de nuestras consultas sobre un mapa haciendo uso de QGIS, como indica la figura B.3. Para ello, es necesario:

1. Ejecutar QGIS y esperar a que se inicie.
2. En el menú de la izquierda, hacer click derecho sobre “PostgreSQL”.
3. Introducir los siguientes datos de conexión:
 - Name: **GTFS**
 - Host: **127.0.0.1**
 - Port: **5432**
 - Database: **gtfs**
 - SSL mode: **prefer**
 - Authentication: **Basic**
 - User name: **postgres** (marcar **store**)
 - Password: **12345678** (marcar **store**)
4. Comprobar que los parámetros de conexión se han introducido correctamente pulsando en “Test Connection” y, a continuación guardar la conexión pulsando “OK”.
5. En el menú de la aplicación, seleccionar *Web > QuickMapServices > SearchQMS*.
6. Buscar, en el menú que se acaba de abrir a la derecha, un proveedor de mapas (se recomienda “OpenStreetMap”), y hacer doble click en él.
7. Tras abrirse el mapa de mundo, hacer zoom sobre la ciudad o región asociada al dataset.
8. Hacer click derecho sobre la conexión que añadimos con anterioridad y pulsar “Execute SQL...”

9. Introducir nuestra consulta SQL y lanzarla pulsando en “Execute”.
10. Una vez aparezcan los resultados, pulsar en “Load as a new layer”, abajo a la izquierda.
11. Marcar la opción “Geometry column” y seleccionar en el desplegable la columna resultado que contenga la geometría que queremos visualizar (contiene un código alfanumérico de 50 caracteres).
12. Añadir la capa pulsando “Load Layer”, y asegurarnos de que queda por encima de la capa de OpenStreetMap, en el apartado “Layers”.
13. Para encontrar en el mapa los resultados que se han vuelto visibles, hacer click derecho en la capa del resultado, “QueryLayer”, y pulsar “Zoom to Layer(s)”.
14. Si se quiere conservar el mapa y los resultados de consultas, guardar el proyecto, ya sea desde el menú principal de la aplicación, o desde el pop-up que aparece al tratar de cerrarla.

B.5.2. Folium (Python)

Otra de las opciones para visualizar los resultados es generar gráficas o mapas interactivos, como el de la figura B.4, utilizando los scripts de Python proporcionados en el directorio **Scripts** del proyecto.

Todos ellos proporcionan detalles sobre sus argumentos si los lanzamos con la opción `-help`. Por ejemplo, con `shortest_path_interactive_map.py`:

options:

<code>-h, --help</code>	show this help message and exit
<code>--origin-stop-id ORIGIN_STOP_ID</code>	The ID of the starting stop.
<code>--destination-stop-id DESTINATION_STOP_ID</code>	The ID of the destination stop.
<code>--date DATE</code>	The departure date in YYYY-MM-DD format.
<code>--time TIME</code>	The departure time in HH:MI:SS format.
<code>--output OUTPUT</code>	Path to save the output map HTML file.

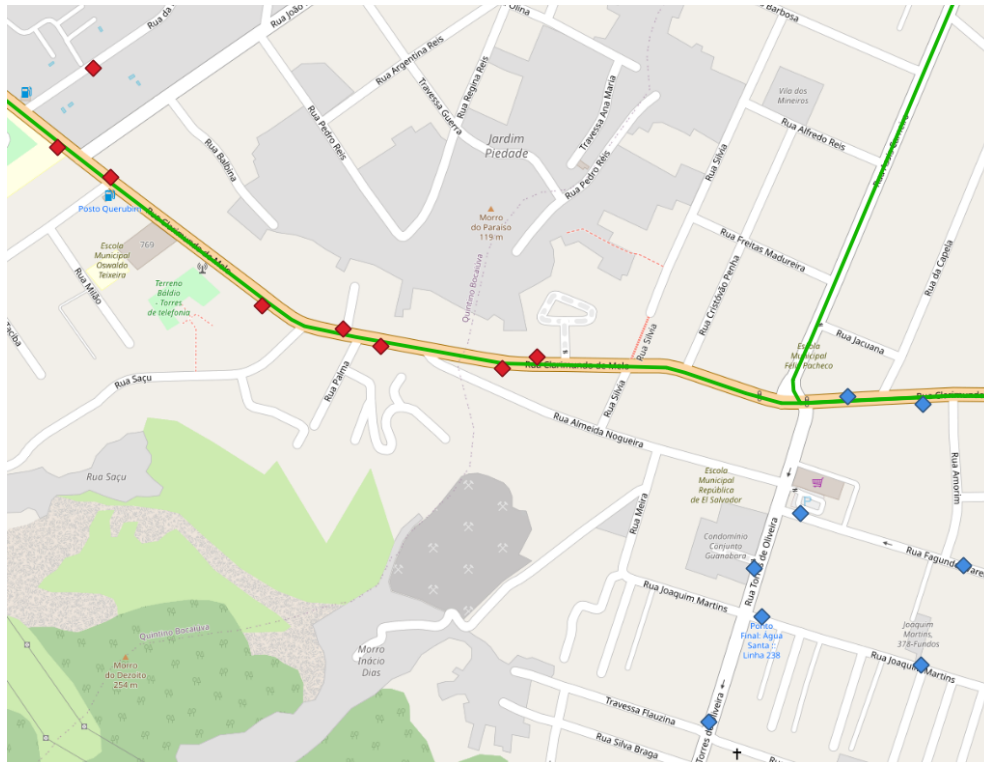


Figura B.3: Visualización de múltiples resultados en QGIS.

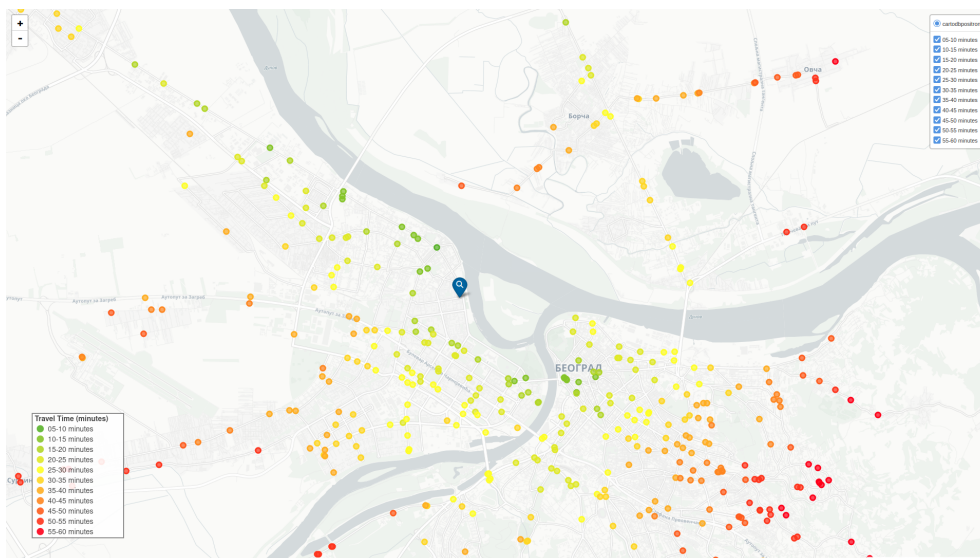


Figura B.4: Visualización de un análisis de alcanzabilidad en Folium.

B.6. Errores frecuentes

B.6.1. No es posible conectar con el daemon de Docker

Este problema se caracteriza por el siguiente mensaje al tratar de lanzar alguno de los sistemas gestores de bases de datos:

```
docker: Cannot connect to the Docker daemon at  
unix:///var/run/docker.sock. Is the docker daemon running?
```

Se nos indica que Docker ha sido instalado correctamente pero el servicio asociado no se ha lanzado. Para solucionarlo, basta con ejecutar el siguiente comando:

```
$ sudo systemctl start docker
```

B.6.2. No se puede establecer comunicación con un contenedor Docker

El mensaje de error que aparece al intentar importar un conjunto de datos es el siguiente:

```
Error response from daemon: No such container: <postgres/neo4j>
```

Al no haberse lanzado el contenedor, el script de importación es incapaz de encontrarlo. Para solucionarlo, desde la carpeta raíz del proyecto, ejecutamos el script `launch.sh` del sistema gestor de bases de datos correspondiente:

```
$ Scripts/<PostgreSQL/Neo4J>/launch.sh
```

B.6.3. Las consultas en Neo4J no producen resultados

Si al tratar de lanzar una consulta en la interfaz web de Neo4J no se nos muestra ningún nodo como resultado, pero tampoco aparece ningún mensaje de error, es posible que estemos utilizando la base de datos por defecto, `neo4j`, en lugar de la que contiene los datos, llamada `gtfs`.

Para cambiar de base de datos, abrimos la sección “Database Information”, correspondiente al primer icono en la barra lateral de la izquierda, y, bajo el texto “Use database”, en el desplegable, elegimos `gtfs`.

B.6.4. Los scripts en Python fallan al tratar de importar bibliotecas

En este caso, al tratar de lanzar un script de importación o generación de mapas interactivos, nos aparece un mensaje de error similar al siguiente (puede mencionar otra biblioteca, no necesariamente la de Neo4J):

```
ModuleNotFoundError: No module named 'neo4j'
```

Como solución, debemos ejecutar los siguientes comandos para lanzar el entorno virtual de Python y obtener las dependencias:

```
$ python3 -m venv .venv
$ source .venv/bin/activate
$ pip install -r requirements.txt
```


Apéndice C

Licencia

La parte documental de este trabajo cuenta con la licencia Creative Commons Attribution-ShareAlike 4.0.

A su vez, todo el código fuente se proporciona bajo la licencia GPLv3. Se incluye una copia íntegra de dicha licencia en el repositorio.

Bibliografía

- [1] E. W. Dijkstra, “A note on two problems in connexion with graphs”, *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [2] Julian Dibbelt, Thomas Pajor, Ben Strasser, and Dorothea Wagner. 2018. Connection Scan Algorithm. *ACM J. Exp. Algorithmics* 23, Artículo 1.7 (2018), 56 páginas. <https://doi.org/10.1145/3274661>
- [3] MobilityData, General Transit Feed Specification (GTFS) Schedule Reference. 2025. <https://gtfs.org/schedule/reference>
- [4] MobilityData, General Transit Feed Specification (GTFS) Realtime Reference. 2025. <https://gtfs.org/realtime/reference>
- [5] The PostgreSQL Global Development Group, PostgreSQL 17 Documentation. 2025. <https://www.postgresql.org/docs/17>
- [6] PostGIS Project Steering Committee, PostGIS 3.5 Manual. 2025. <https://postgis.net/docs/manual-3.5>
- [7] Neo4j, Inc., Neo4j Graph Database Operations Manual, v5, 2025. <https://neo4j.com/docs/operations-manual/5>
- [8] Neo4j, Inc., The Cypher Query Language Reference, v5, 2025. <https://neo4j.com/docs/cypher-manual/5>
- [9] Neo4j, Inc., APOC Extended Documentation, v5, 2025. <https://neo4j.com/labs/apoc/5>
- [10] Neo4j, Inc., Neo4j-Spatial User Guide, v5, 2025. <https://neo4j.com/labs/neo4j-spatial/5>
- [11] Docker Inc., Docker Engine User Guide. 2025. <https://docs.docker.com/engine>
- [12] Python Software Foundation, Python 3.13 Documentation. 2025. <https://docs.python.org/3.13>

- [13] H. Krekel et al., pytest: helps you write better programs, v8.4.x, 2025. <https://docs.pytest.org/en/8.4.x>
- [14] D. R. MacIver et al., Hypothesis: Property-based testing for Python. 2025. <https://hypothesis.readthedocs.io>
- [15] Folium Developers, Folium: Python data, leaflet.js maps, v0.20.0, 2025. <https://python-visualization.github.io/folium>
- [16] The pandas development team, pandas documentation, v2.3, 2025. <https://pandas.pydata.org/pandas-docs/version/2.3>
- [17] QGIS Development Team, QGIS User Guide, v3.40, 2025. <https://www.qgis.org/resources/hub>
- [18] Wikipedia, Betweenness centrality. Artículo de wikipedia (https://en.wikipedia.org/wiki/Betweenness_centrality). Consultado en noviembre de 2025.